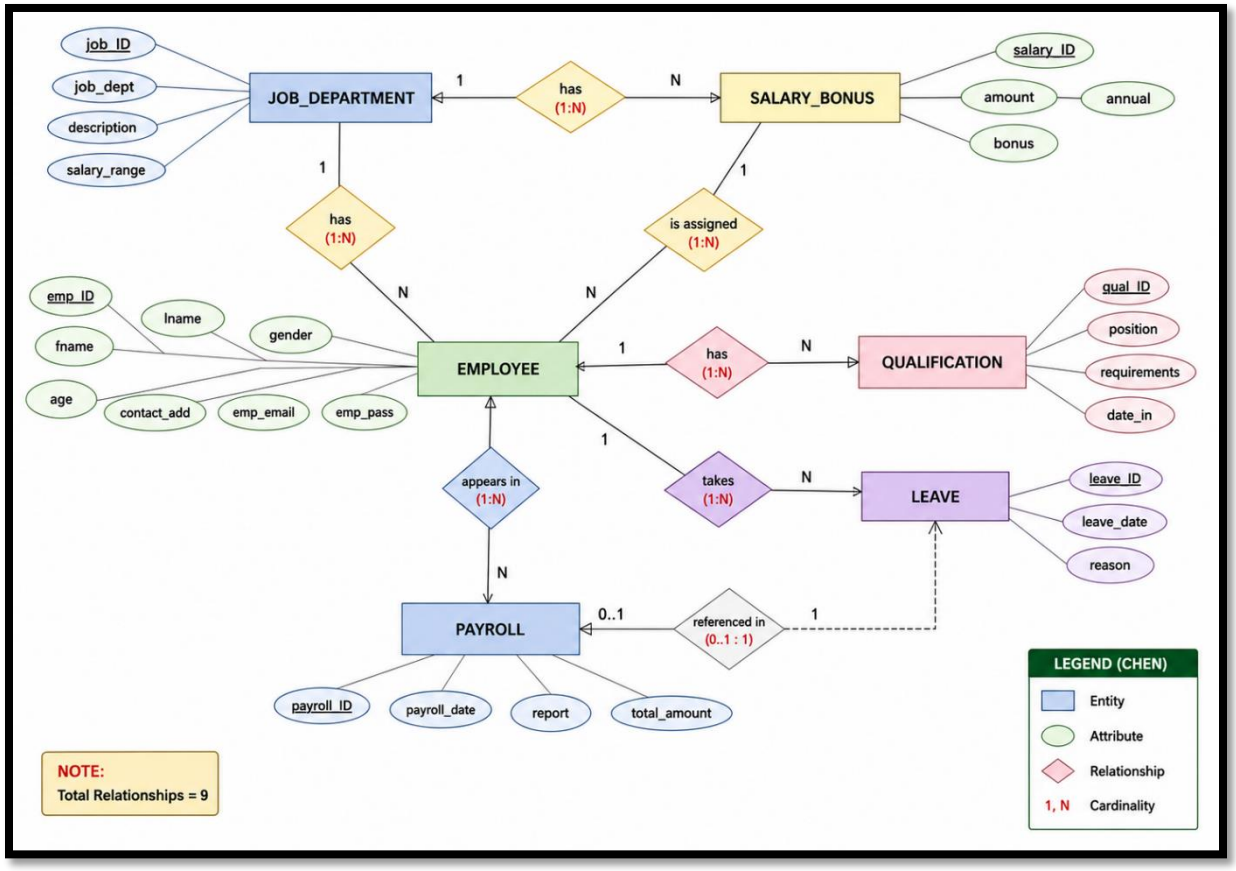
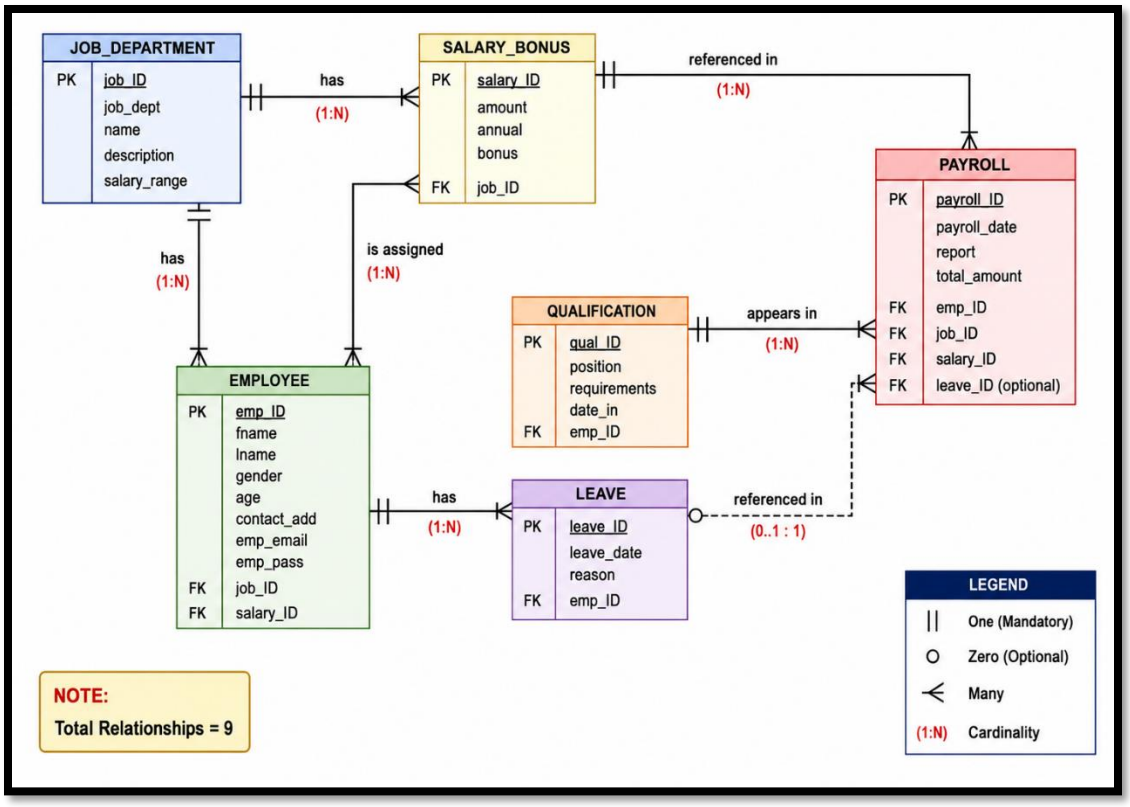
EMPLOYEE MANAGEMENT SYSTEM

Database Fundamentals — Capstone Project PL/SQL · Oracle Database Covers: ERD · Mapping · Normalization · SQL DML · Aggregation · Joins · Subqueries · Views · Stored Procedures · Triggers · Scheduler Jobs

ERP:

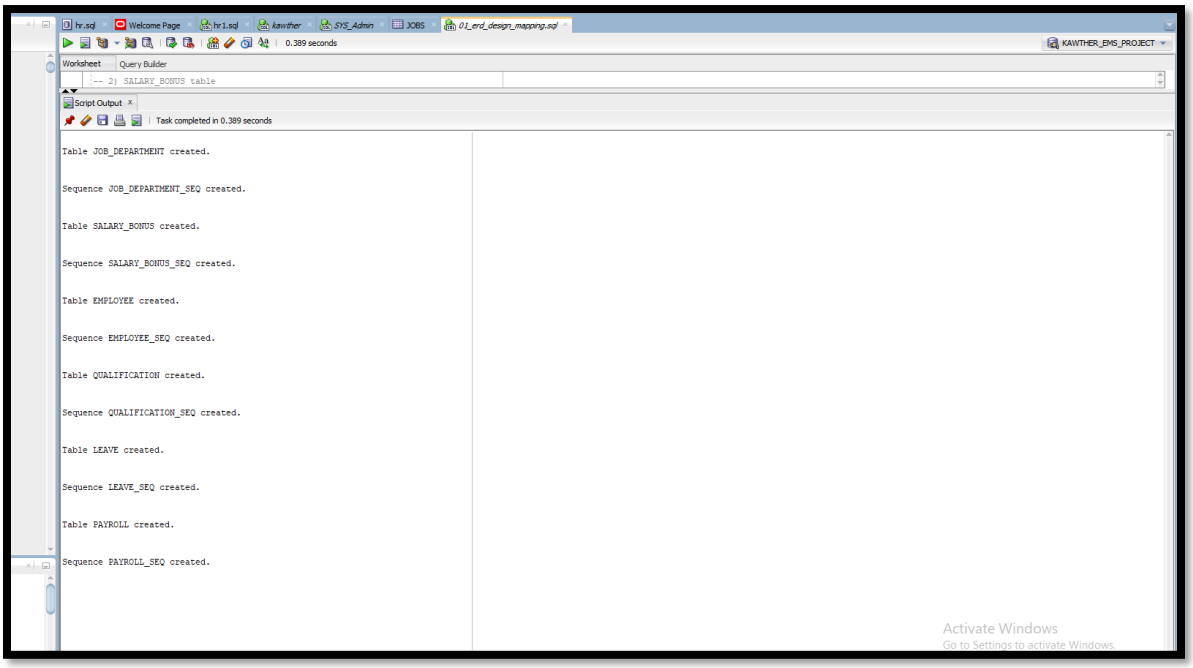


Crow's Foot (IE) notation:

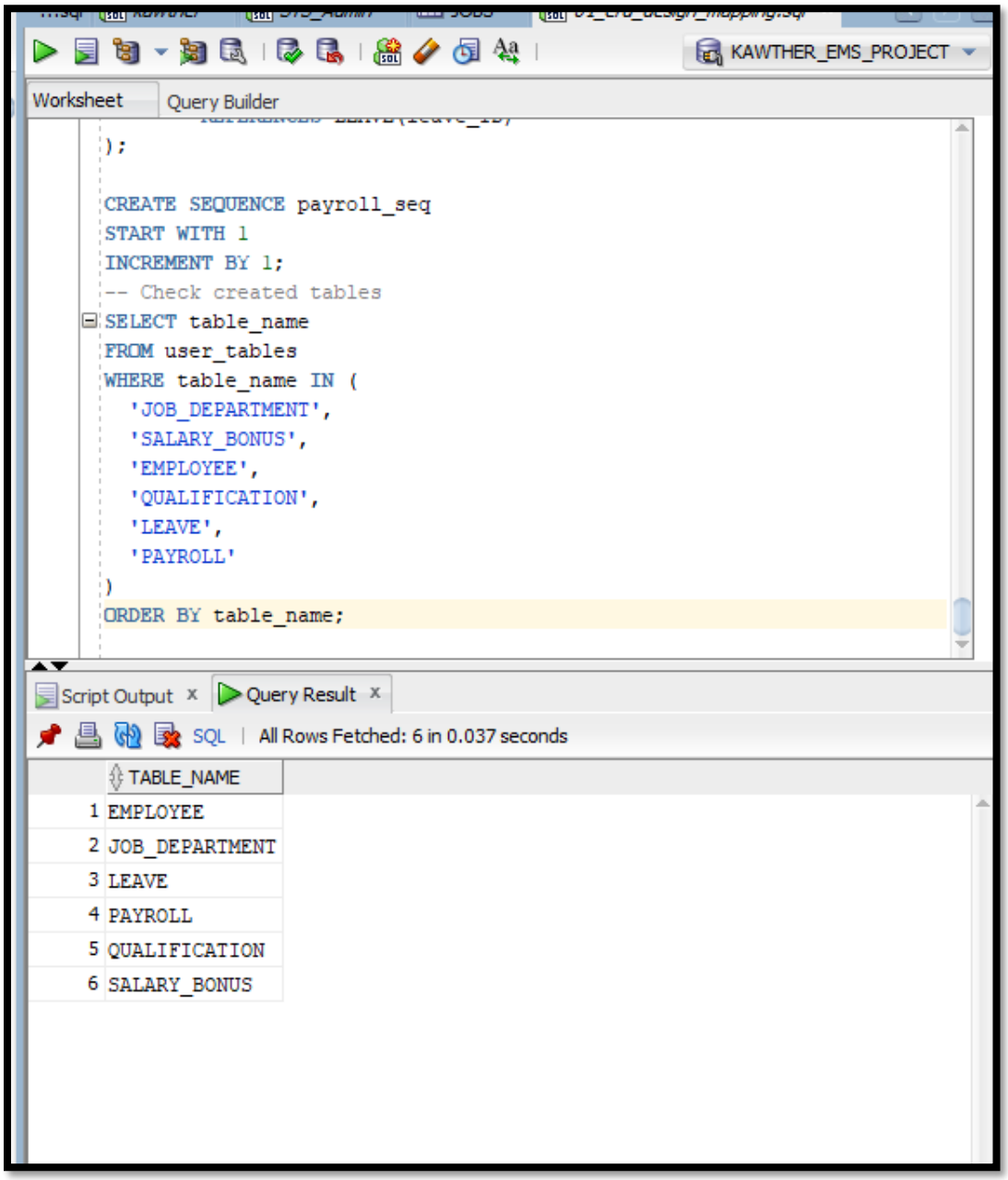


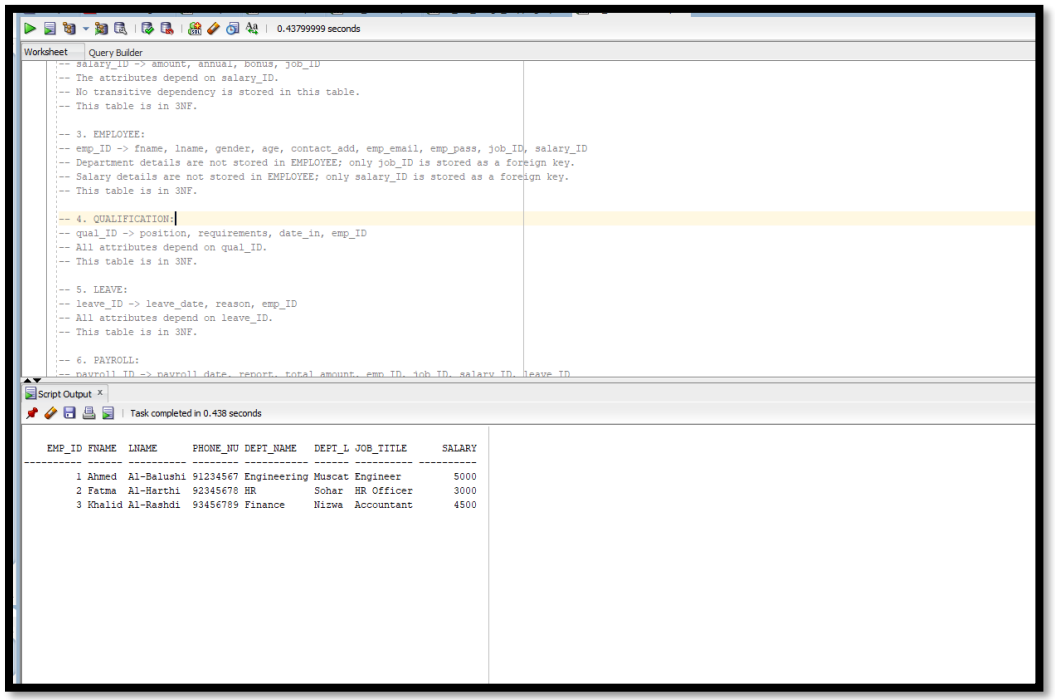
01_erd_design_mapping.sql:

Section 01 - Task 3: CREATE TABLE + SEQUENCE:

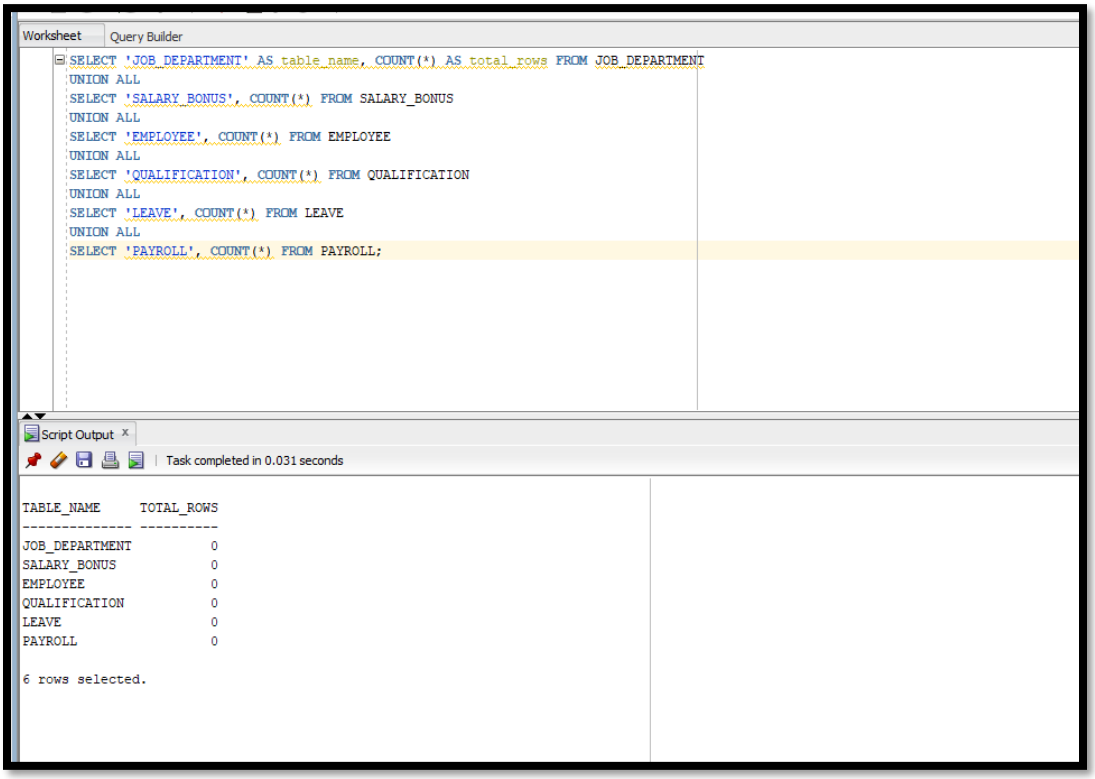


DDL: CREATE table with relationship :


KAWTHER_EMS_PROJECT
Worksheet
Query Builder
);
CREATE SEQUENCE payroll_seq
START WITH 1
INCREMENT BY 1;
-- Check created tables
SELECT table_name
FROM user_tables
WHERE table_name IN (
'JOB_DEPARTMENT',
'SALARY_BONUS',
'EMPLOYEE',
'QUALIFICATION',
'LEAVE',
'PAYROLL'
)
ORDER BY table_name;
Script Output
Query Result
SQL
All Rows Fetched: 6 in 0.037 seconds
TABLE_NAME
1 EMPLOYEE
2 JOB_DEPARTMENT
3 LEAVE
4 PAYROLL
5 QUALIFICATION
6 SALARY_BONUS



03_dml.sql — All INSERT / UPDATE / DELETE / SELECT tasks:



```
-- ================================================
-- SECTION 03 - TASK 1
-- Step 1: Insert JOB_DEPARTMENT records
-- ================================================

INSERT INTO JOB_DEPARTMENT VALUES
(job_department_seq.NEXTVAL, 'Engineering', 'Engineering Department', 'Handles technical projects', '3000-8000');

INSERT INTO JOB_DEPARTMENT VALUES
(job_department_seq.NEXTVAL, 'HR', 'Human Resources', 'Manages employees and recruitment', '1500-4000');

INSERT INTO JOB_DEPARTMENT VALUES
(job_department_seq.NEXTVAL, 'Finance', 'Finance Department', 'Handles payroll and budgets', '2500-7000');

INSERT INTO JOB_DEPARTMENT VALUES
(job_department_seq.NEXTVAL, 'Maintenance', 'Maintenance Department', 'Responsible for repairs and support', '1000-3500');

INSERT INTO JOB_DEPARTMENT VALUES
(job_department_seq.NEXTVAL, 'IT', 'Information Technology', 'Manages systems and networks', '2500-7500');

-- Check departments
SELECT * FROM JOB_DEPARTMENT;
```

Script Output | Task completed in 0.141 seconds

5 rows inserted.

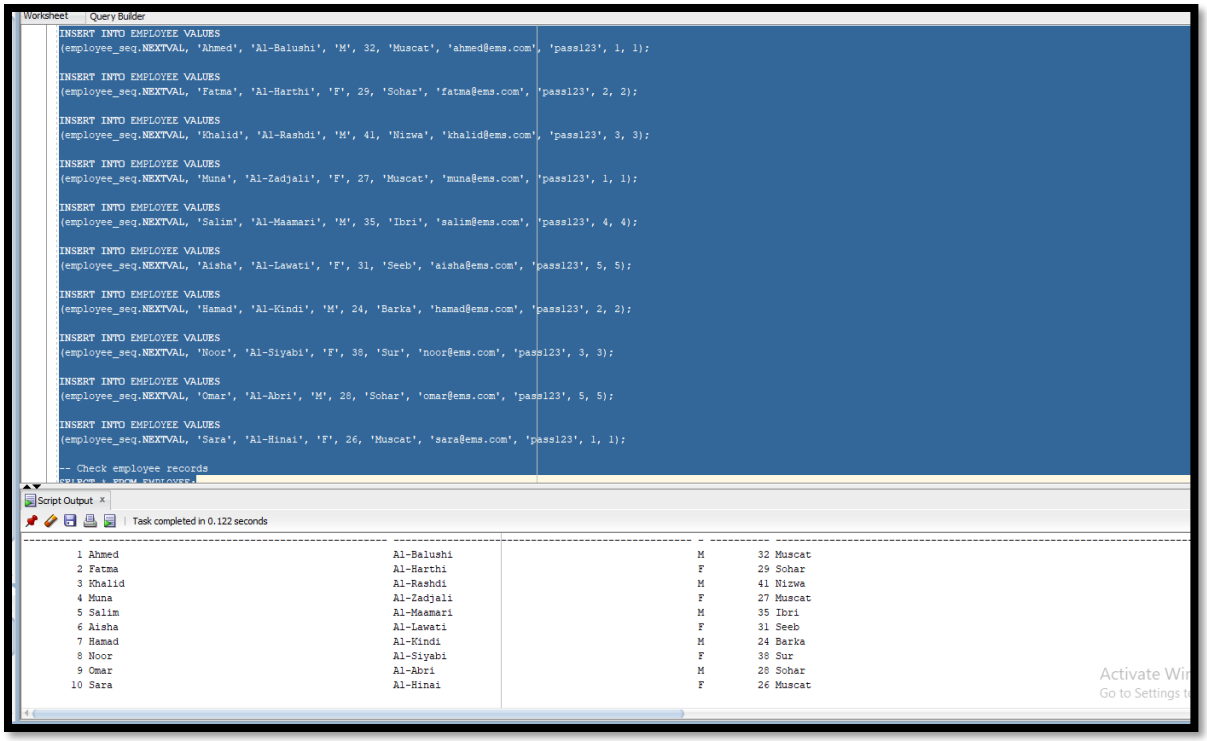
JOB_ID	JOB_DEPT	NAME	DESCRIPTION
1	Engineering	Engineering Department	Handles technical projects
2	HR	Human Resources	Manages employees and recruitment
3	Finance	Finance Department	Handles payroll and budgets
4	Maintenance	Maintenance Department	Responsible for repairs and support
5	IT	Information Technology	Manages systems and networks


```
INSERT INTO SALARY_BONUS VALUES
(salary_bonus_seq.NEXTVAL, 5000, 60000, 700, 1);

INSERT INTO SALARY_BONUS VALUES
(salary_bonus_seq.NEXTVAL, 3000, 36000, 400, 2);

INSERT INTO SALARY_BONUS VALUES
(salary_bonus_seq.NEXTVAL, 4500, 54000, 600, 3);

INSERT INTO SALARY_BONUS VALUES
(salary_bonus_seq.NEXTVAL, 2500, 30000, 300, 4);

INSERT INTO SALARY_BONUS VALUES
(salary_bonus_seq.NEXTVAL, 5500, 66000, 800, 5);

-- Check salary bonus records
SELECT * FROM SALARY_BONUS;
```

Script Output | Task completed in 0.074 seconds

5 rows inserted.

SALARY_ID	AMOUNT	ANNUAL	BONUS	JOB_ID
1	5000	60000	700	1
2	3000	36000	400	2
3	4500	54000	600	3
4	2500	30000	300	4
5	5500	66000	800	5

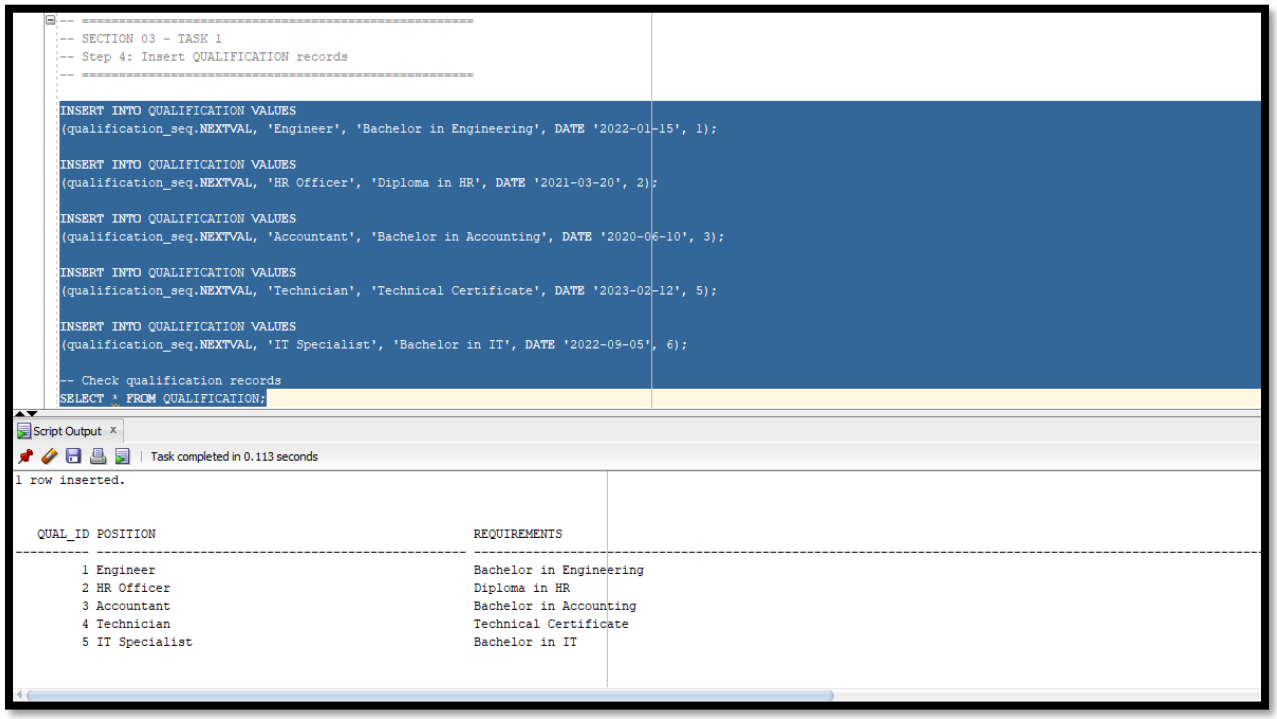


```
Worksheet  Query Builder

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Ahmed', 'Al-Balushi', 'M', 32, 'Muscat', 'ahmed@ems.com', 'pass123', 1, 1);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Fatma', 'Al-Harthi', 'F', 29, 'Sohar', 'fatma@ems.com', 'pass123', 2, 2);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Khalid', 'Al-Rashdi', 'M', 41, 'Nizwa', 'khalid@ems.com', 'pass123', 3, 3);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Muna', 'Al-Zadjali', 'F', 27, 'Muscat', 'muna@ems.com', 'pass123', 1, 1);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Salim', 'Al-Maamari', 'M', 35, 'Ibri', 'salim@ems.com', 'pass123', 4, 4);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Aisha', 'Al-Lawati', 'F', 31, 'Seeb', 'aisha@ems.com', 'pass123', 5, 5);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Hamad', 'Al-Kindi', 'M', 24, 'Barka', 'hamad@ems.com', 'pass123', 2, 2);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Noor', 'Al-Siyabi', 'F', 38, 'Sur', 'noor@ems.com', 'pass123', 3, 3);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Omar', 'Al-Abri', 'M', 28, 'Sohar', 'omar@ems.com', 'pass123', 5, 5);

INSERT INTO EMPLOYEE VALUES
(employee_seq.NEXTVAL, 'Sara', 'Al-Hinai', 'F', 26, 'Muscat', 'sara@ems.com', 'pass123', 1, 1);

-- Check employee records
SELECT * FROM EMPLOYEE;
```

Script Output

Task completed in 0.122 seconds

1	Ahmed	Al-Balushi	M	32	Muscat	ahmed@ems.com	pass123	1
2	Fatma	Al-Harthi	F	29	Sohar	fatma@ems.com	pass123	2
3	Khalid	Al-Rashdi	M	41	Nizwa	khalid@ems.com	pass123	3
4	Muna	Al-Zadjali	F	27	Muscat	muna@ems.com	pass123	1
5	Salim	Al-Maamari	M	35	Ibri	salim@ems.com	pass123	4
6	Aisha	Al-Lawati	F	31	Seeb	aisha@ems.com	pass123	5
7	Hamad	Al-Kindi	M	24	Barka	hamad@ems.com	pass123	2
8	Noor	Al-Siyabi	F	38	Sur	noor@ems.com	pass123	3
9	Omar	Al-Abri	M	28	Sohar	omar@ems.com	pass123	5
10	Sara	Al-Hinai	F	26	Muscat	sara@ems.com	pass123	1



```
-- ==================================================
-- SECTION 03 - TASK 1
-- Step 4: Insert QUALIFICATION records
-- ==================================================

INSERT INTO QUALIFICATION VALUES
(qualification_seq.NEXTVAL, 'Engineer', 'Bachelor in Engineering', DATE '2022-01-15', 1);

INSERT INTO QUALIFICATION VALUES
(qualification_seq.NEXTVAL, 'HR Officer', 'Diploma in HR', DATE '2021-03-20', 2);

INSERT INTO QUALIFICATION VALUES
(qualification_seq.NEXTVAL, 'Accountant', 'Bachelor in Accounting', DATE '2020-06-10', 3);

INSERT INTO QUALIFICATION VALUES
(qualification_seq.NEXTVAL, 'Technician', 'Technical Certificate', DATE '2023-02-12', 5);

INSERT INTO QUALIFICATION VALUES
(qualification_seq.NEXTVAL, 'IT Specialist', 'Bachelor in IT', DATE '2022-09-05', 6);

-- Check qualification records
SELECT * FROM QUALIFICATION;
```

Script Output

Task completed in 0.113 seconds

```
1 row inserted.
```

QUAL_ID	POSITION	REQUIREMENTS
1	Engineer	Bachelor in Engineering
2	HR Officer	Diploma in HR
3	Accountant	Bachelor in Accounting
4	Technician	Technical Certificate
5	IT Specialist	Bachelor in IT

```
INSERT INTO LEAVE VALUES
(leave_seq.NEXTVAL, DATE '2024-01-10', 'Sick leave', 1);

INSERT INTO LEAVE VALUES
(leave_seq.NEXTVAL, DATE '2024-02-15', 'Annual leave', 2);

INSERT INTO LEAVE VALUES
(leave_seq.NEXTVAL, DATE '2024-03-18', 'Emergency leave', 4);

INSERT INTO LEAVE VALUES
(leave_seq.NEXTVAL, DATE '2023-12-05', 'Sick leave', 6);

INSERT INTO LEAVE VALUES
(leave_seq.NEXTVAL, DATE '2024-04-22', 'Family reason', 8);

-- Check leave records
SELECT * FROM LEAVE;
```

Script Output

Task completed in 0.102 seconds

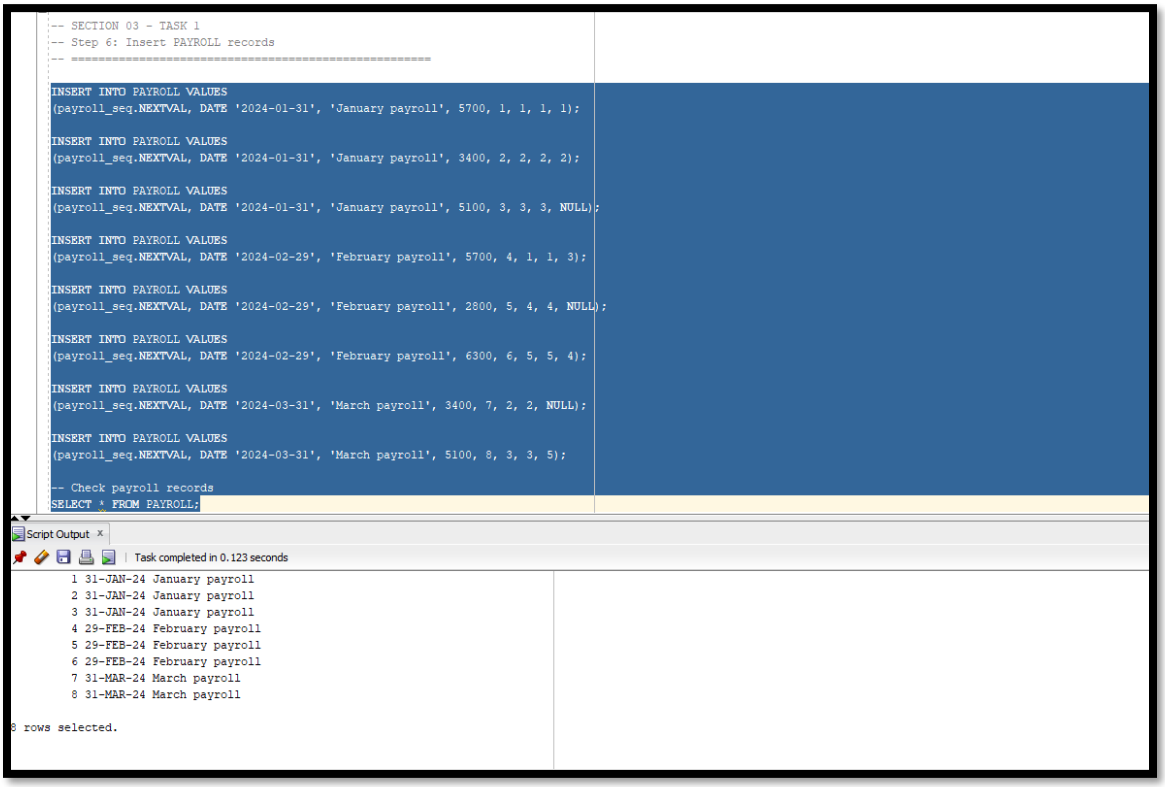
```
1 row inserted.


  LEAVE_ID LEAVE_DAT REASON
---------- --------- --------------------------------------------------------------------------------------------------------------------------------
         1 10-JAN-24 Sick leave
         2 15-FEB-24 Annual leave
         3 18-MAR-24 Emergency leave
         4 05-DEC-23 Sick leave
         5 22-APR-24 Family reason
```

```
SELECT 'JOB_DEPARTMENT' AS table_name, COUNT(*) AS total_rows FROM JOB_DEPARTMENT
UNION ALL
SELECT 'SALARY_BONUS', COUNT(*) FROM SALARY_BONUS
UNION ALL
SELECT 'EMPLOYEE', COUNT(*) FROM EMPLOYEE
UNION ALL
SELECT 'QUALIFICATION', COUNT(*) FROM QUALIFICATION
UNION ALL
SELECT 'LEAVE', COUNT(*) FROM LEAVE
UNION ALL
SELECT 'PAYROLL', COUNT(*) FROM PAYROLL;
```

Script Output

Task completed in 0.06 seconds

```
TABLE_NAME     TOTAL_ROWS
-------------- ----------
JOB_DEPARTMENT          5
SALARY_BONUS            5
EMPLOYEE               10
QUALIFICATION           5
LEAVE                   5
PAYROLL                 8

6 rows selected.
```



```
-- SECTION 03 - TASK 1
-- Step 6: Insert PAYROLL records
-- ================================================

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-01-31', 'January payroll', 5700, 1, 1, 1, 1);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-01-31', 'January payroll', 3400, 2, 2, 2, 2);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-01-31', 'January payroll', 5100, 3, 3, 3, NULL);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-02-29', 'February payroll', 5700, 4, 1, 1, 3);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-02-29', 'February payroll', 2800, 5, 4, 4, NULL);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-02-29', 'February payroll', 6300, 6, 5, 5, 4);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-03-31', 'March payroll', 3400, 7, 2, 2, NULL);

INSERT INTO PAYROLL VALUES
(payroll_seq.NEXTVAL, DATE '2024-03-31', 'March payroll', 5100, 8, 3, 3, 5);

-- Check payroll records
SELECT * FROM PAYROLL;
```

Script Output

Task completed in 0.123 seconds

```
         1 31-JAN-24 January payroll
         2 31-JAN-24 January payroll
         3 31-JAN-24 January payroll
         4 29-FEB-24 February payroll
         5 29-FEB-24 February payroll
         6 29-FEB-24 February payroll
         7 31-MAR-24 March payroll
         8 31-MAR-24 March payroll

8 rows selected.
```

```
COMMIT;
```

Script Output | Task completed in 0.047 seconds

```
SALARY_BONUS              5
EMPLOYEE                 10
QUALIFICATION             5
LEAVE                     5
PAYROLL                   8

6 rows selected.


Commit complete.
```

```
-- ======================================================
-- SECTION 03 - TASK 2
-- Conditional SELECT Queries
-- ======================================================

-- (a) List all employees whose age is between 25 and 40,
-- ordered by last name ascending.

SELECT emp_ID,
       fname || ' ' || lname AS full_name,
       gender,
       age,
       contact_add
FROM EMPLOYEE
WHERE age BETWEEN 25 AND 40
ORDER BY lname ASC;
```

Script Output | Query Result | SQL | All Rows Fetched: 8 in 0.01 seconds

	EMP_ID	FULL_NAME	GENDER	AGE	CONTACT_ADD
1	9	Omar Al-Abri	M	28	Sohar
2	1	Ahmed Al-Balushi	M	32	Muscat
3	2	Fatma Al-Harthi	F	29	Sohar
4	10	Sara Al-Hinai	F	26	Muscat
5	6	Aisha Al-Lawati	F	31	Seeb
6	5	Salim Al-Maamari	M	35	Ibri
7	8	Noor Al-Siyabi	F	38	Sur
8	4	Muna Al-Zadjali	F	27	Muscat

```sql
-- (b) Retrieve all payroll records where total_amount exceeds 5000,
-- showing employee name and department.

SELECT p.payroll_ID,
       e.fname || ' ' || e.lname AS employee_name,
       d.name AS department_name,
       p.payroll_date,
       p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e
    ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT d
    ON p.job_ID = d.job_ID
WHERE p.total_amount > 5000
ORDER BY p.total_amount DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 5 in 0.008 seconds

	PAYROLL_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	PAYROLL_DATE	TOTAL_AMOUNT
1	6	Aisha Al-Lawati	Information Technology	29-FEB-24	6300
2	1	Ahmed Al-Balushi	Engineering Department	31-JAN-24	5700
3	4	Muna Al-Zadjali	Engineering Department	29-FEB-24	5700
4	3	Khalid Al-Rashdi	Finance Department	31-JAN-24	5100
5	8	Noor Al-Siyabi	Finance Department	31-MAR-24	5100

```sql
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS employee_name,
       l.leave_date,
       l.reason
FROM EMPLOYEE e
JOIN LEAVE l
    ON e.emp_ID = l.emp_ID
WHERE LOWER(l.reason) LIKE '%sick%';
```

Script Output | Query Result

SQL | All Rows Fetched: 2 in 0.01 seconds

	EMP_ID	EMPLOYEE_NAME	LEAVE_DATE	REASON
1	1	Ahmed Al-Balushi	10-JAN-24	Sick leave
2	6	Aisha Al-Lawati	05-DEC-23	Sick leave

```sql
SELECT d.job_ID,
       d.name AS department_name,
       d.job_dept
FROM JOB_DEPARTMENT d
WHERE NOT EXISTS (
    SELECT 1
    FROM EMPLOYEE e
    WHERE e.job_ID = d.job_ID
);
```

Script Output | Query Result

SQL | All Rows Fetched: 0 in 0.004 seconds

JOB_ID	DEPARTM...	JOB_DEPT

Update:

```
       d.job_dept,
       sb.amount,
       sb.annual,
       sb.bonus
FROM SALARY_BONUS sb
JOIN JOB_DEPARTMENT d
    ON sb.job_ID = d.job_ID
WHERE d.job_dept = 'Engineering';


UPDATE SALARY_BONUS sb
SET sb.amount = sb.amount * 1.10,
    sb.annual = sb.annual * 1.10
WHERE sb.salary_ID IN (
    SELECT DISTINCT e.salary_ID
    FROM EMPLOYEE e
    JOIN JOB_DEPARTMENT d
        ON e.job_ID = d.job_ID
    WHERE d.job_dept = 'Engineering'
);


-- Verify Engineering salary after update
SELECT sb.salary_ID,
       d.job_dept,
       sb.amount,
       sb.annual,
       sb.bonus
FROM SALARY_BONUS sb
JOIN JOB_DEPARTMENT d
    ON sb.job_ID = d.job_ID
WHERE d.job_dept = 'Engineering';
```

Script Output | Query Result | Query Result 1

SQL | All Rows Fetched: 1 in 0.003 seconds

	SALARY_ID	JOB_DEPT	AMOUNT	ANNUAL	BONUS
1	1	Engineering	5500	66000	700

```
-- (b) Update the emp_email of all employees to lowercase
-- using Oracle LOWER() function.

-- Preview emails before update
SELECT emp_ID,
       fname || ' ' || lname AS employee_name,
       emp_email
FROM EMPLOYEE;


UPDATE EMPLOYEE
SET emp_email = LOWER(emp_email);


-- Verify emails after update
SELECT emp_ID,
       fname || ' ' || lname AS employee_name,
       emp_email
FROM EMPLOYEE;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 10 in 0.002 seconds

	EMP_ID	EMPLOYEE_NAME	EMP_EMAIL
1	1	Ahmed Al-Balushi	ahmed@ems.com
2	2	Fatma Al-Harthi	fatma@ems.com
3	3	Khalid Al-Rashdi	khalid@ems.com
4	4	Muna Al-Zadjali	muna@ems.com
5	5	Salim Al-Maamari	salim@ems.com
6	6	Aisha Al-Lawati	aisha@ems.com
7	7	Hamad Al-Kindi	hamad@ems.com
8	8	Noor Al-Siyabi	noor@ems.com
9	9	Omar Al-Abri	omar@ems.com
10	10	Sara Al-Hinai	sara@ems.com

```sql
-- Preview departments before update
SELECT d.job_ID,
       d.job_dept,
       d.name,
       d.salary_range
FROM JOB_DEPARTMENT d;


UPDATE JOB_DEPARTMENT d
SET d.salary_range = 'REVISED'
WHERE d.job_ID IN (
    SELECT p.job_ID
    FROM PAYROLL p
    GROUP BY p.job_ID
    HAVING AVG(p.total_amount) > 8000
);


-- Verify departments after update
SELECT d.job_ID,
       d.job_dept,
       d.name,
       d.salary_range
FROM JOB_DEPARTMENT d;
-- Note: No department was updated because no department average payroll exceeds 8000.
COMMIT;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

Task completed in 0.045 seconds

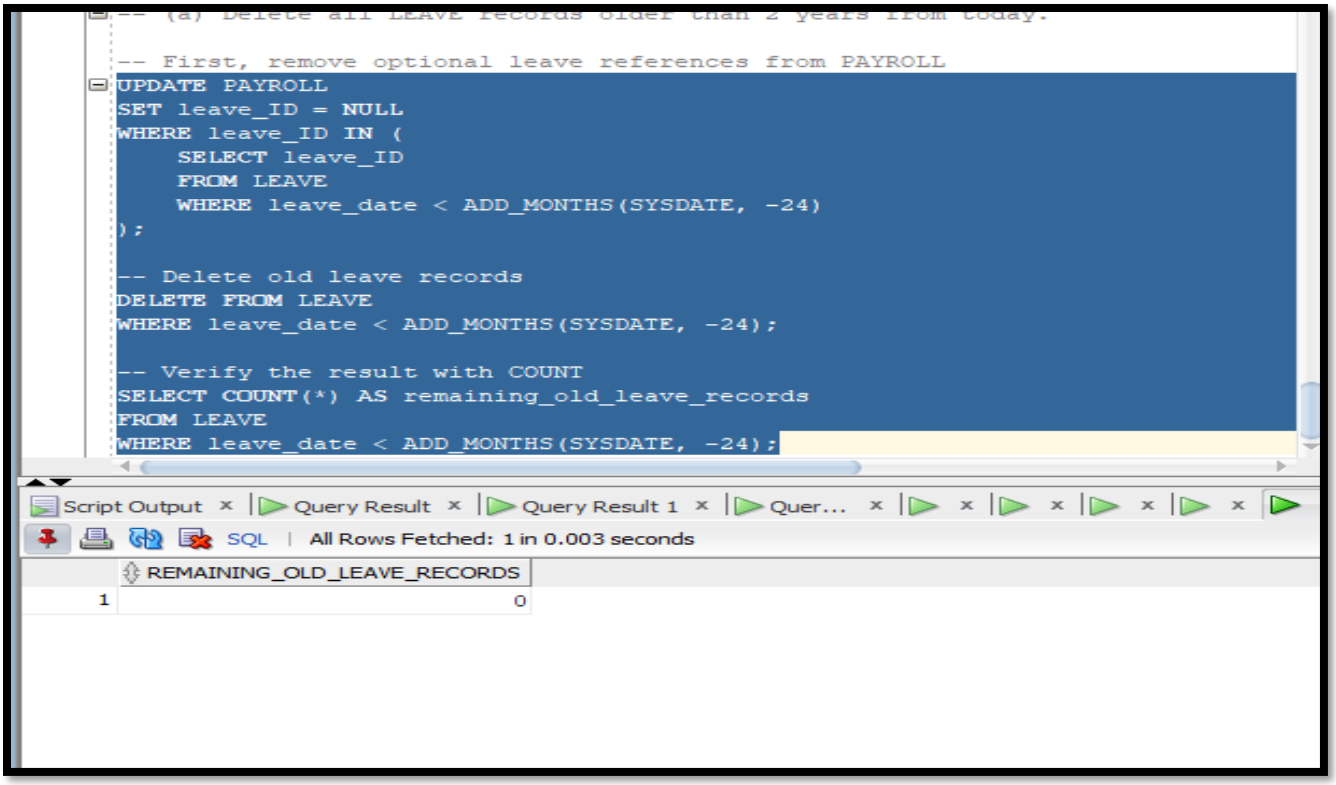
```
10 rows updated.

>>Query Run In:Query Result 3

0 rows updated.

>>Query Run In:Query Result 5

Commit complete.
```

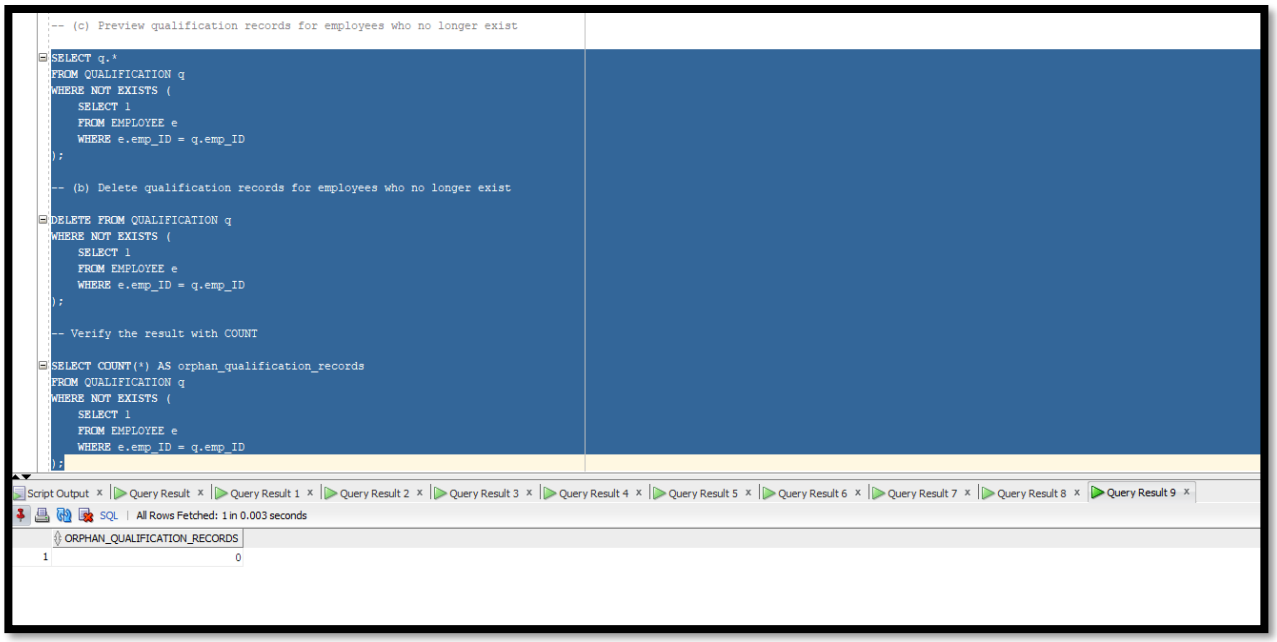
```sql
-- ============================================
-- SECTION 03 - TASK 4
-- Controlled DELETE with Safety Checks
-- ============================================

SAVEPOINT before_delete_task;

-- Preview leave records older than 2 years from today
SELECT *
FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);
```

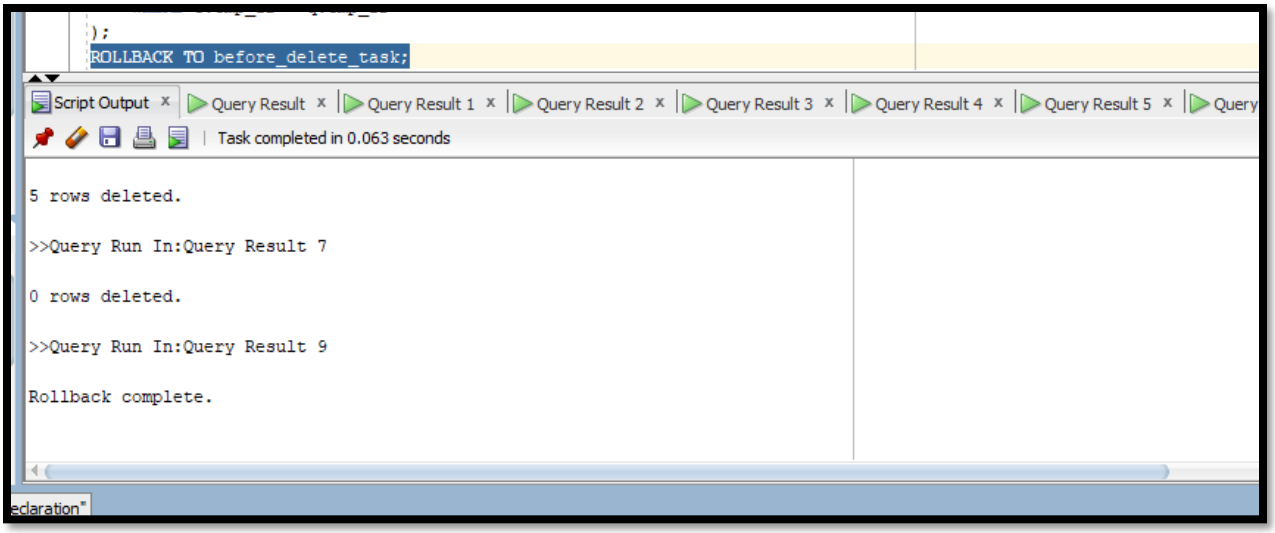
Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6

SQL | All Rows Fetched: 5 in 0.007 seconds

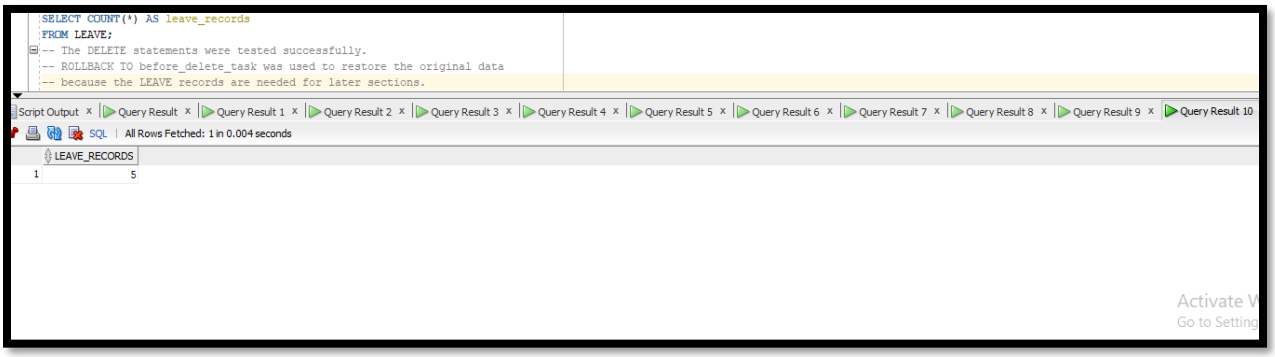
	LEAVE_ID	LEAVE_DATE	REASON	EMP_ID
1	1	10-JAN-24	Sick leave	1
2	2	15-FEB-24	Annual leave	2
3	3	18-MAR-24	Emergency leave	4
4	4	05-DEC-23	Sick leave	6
5	5	22-APR-24	Family reason	8


-- (a) Delete all LEAVE records older than 2 years from today.
-- First, remove optional leave references from PAYROLL
UPDATE PAYROLL
SET leave_ID = NULL
WHERE leave_ID IN (
SELECT leave_ID
FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24)
);
-- Delete old leave records
DELETE FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);
-- Verify the result with COUNT
SELECT COUNT(*) AS remaining_old_leave_records
FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);
Script Output
Query Result
Query Result 1
Quer...
SQL
All Rows Fetched: 1 in 0.003 seconds
REMAINING_OLD_LEAVE_RECORDS
1
0


-- (c) Preview qualification records for employees who no longer exist
SELECT q.*
FROM QUALIFICATION q
WHERE NOT EXISTS (
SELECT 1
FROM EMPLOYEE e
WHERE e.emp_ID = q.emp_ID
);
-- (b) Delete qualification records for employees who no longer exist
DELETE FROM QUALIFICATION q
WHERE NOT EXISTS (
SELECT 1
FROM EMPLOYEE e
WHERE e.emp_ID = q.emp_ID
);
-- Verify the result with COUNT
SELECT COUNT(*) AS orphan_qualification_records
FROM QUALIFICATION q
WHERE NOT EXISTS (
SELECT 1
FROM EMPLOYEE e
WHERE e.emp_ID = q.emp_ID
);
Script Output
Query Result
Query Result 1
Query Result 2
Query Result 3
Query Result 4
Query Result 5
Query Result 6
Query Result 7
Query Result 8
Query Result 9
SQL
All Rows Fetched: 1 in 0.003 seconds
ORPHAN_QUALIFICATION_RECORDS
1
0



Rollback



Delete

05_joins.sql — All join queries:

```
SELECT d.name AS department_name,
       COUNT(e.emp_ID) AS total_employees
FROM JOB_DEPARTMENT d
LEFT JOIN EMPLOYEE e
    ON d.job_ID = e.job_ID
GROUP BY d.name
ORDER BY d.name;


-- (b) Minimum, maximum, and average salary amount.

SELECT MIN(amount) AS minimum_salary,
       MAX(amount) AS maximum_salary,
       ROUND(AVG(amount), 2) AS average_salary
FROM SALARY_BONUS;


-- (c) Total bonus paid out across the entire company.

SELECT SUM(bonus) AS total_bonus
```

Script Output | Task completed in 0.074 seconds

```
DEPARTMENT_NAME                                    TOTAL_EMPLOYEES
-------------------------------------------------- ---------------
Engineering Department                                           3
Finance Department                                               2
Human Resources                                                  2
Information Technology                                           2
Maintenance Department                                           1


MINIMUM_SALARY MAXIMUM_SALARY AVERAGE_SALARY
-------------- -------------- --------------
          2500           5500           4200


TOTAL_BONUS
-----------
       2800
```

```
SELECT d.name AS department_name,
       ROUND(AVG(e.age), 2) AS average_age
FROM JOB_DEPARTMENT d
JOIN EMPLOYEE e
    ON d.job_ID = e.job_ID
GROUP BY d.name
HAVING AVG(e.age) > 30;


-- (b) Show all job titles where more than 2 employees share that qualification position.

SELECT q.position,
       COUNT(q.emp_ID) AS total_employees
FROM QUALIFICATION q
GROUP BY q.position
HAVING COUNT(q.emp_ID) > 2;


-- (c) Find months from PAYROLL where the total payroll amount exceeds 20000.

SELECT TO_CHAR(payroll_date, 'YYYY-MM') AS payroll_month,
       SUM(total_amount) AS total_monthly_payroll
FROM PAYROLL
GROUP BY TO_CHAR(payroll_date, 'YYYY-MM')
HAVING SUM(total_amount) > 20000
ORDER BY payroll_month;
```

Script Output | Task completed in 0.061 seconds

```
DEPARTMENT_NAME                                    AVERAGE_AGE
-------------------------------------------------- -----------
Maintenance Department                                      35
Finance Department                                        39.5

no rows selected
no rows selected
```

```sql
-- ======================================================
-- TASK 3: Aggregation with Multiple Functions
-- ======================================================

-- Department summary report:
-- department name, employee count, total payroll,
-- average salary, highest salary, lowest salary.

SELECT d.name AS department_name,
       COUNT(DISTINCT e.emp_ID) AS total_employees,
       NVL(SUM(p.total_amount), 0) AS total_payroll_amount,
       ROUND(AVG(sb.amount), 2) AS average_salary,
       MAX(sb.amount) AS highest_salary,
       MIN(sb.amount) AS lowest_salary
FROM JOB_DEPARTMENT d
LEFT JOIN EMPLOYEE e
    ON d.job_ID = e.job_ID
LEFT JOIN SALARY_BONUS sb
    ON e.salary_ID = sb.salary_ID
LEFT JOIN PAYROLL p
    ON e.emp_ID = p.emp_ID
GROUP BY d.name
ORDER BY total_payroll_amount DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 5 in 0.01 seconds

	DEPARTMENT_NAME	TOTAL_EMPLOYEES	TOTAL_PAYROLL_AMOUNT	AVERAGE_SALARY	HIGHEST_SALARY	LOWEST_SALARY
1	Engineering Department	3	11400	5500	5500	5500
2	Finance Department	2	10200	4500	4500	4500
3	Human Resources	2	6800	3000	3000	3000
4	Information Technology	2	6300	5500	5500	5500
5	Maintenance Department	1	2800	2500	2500	2500

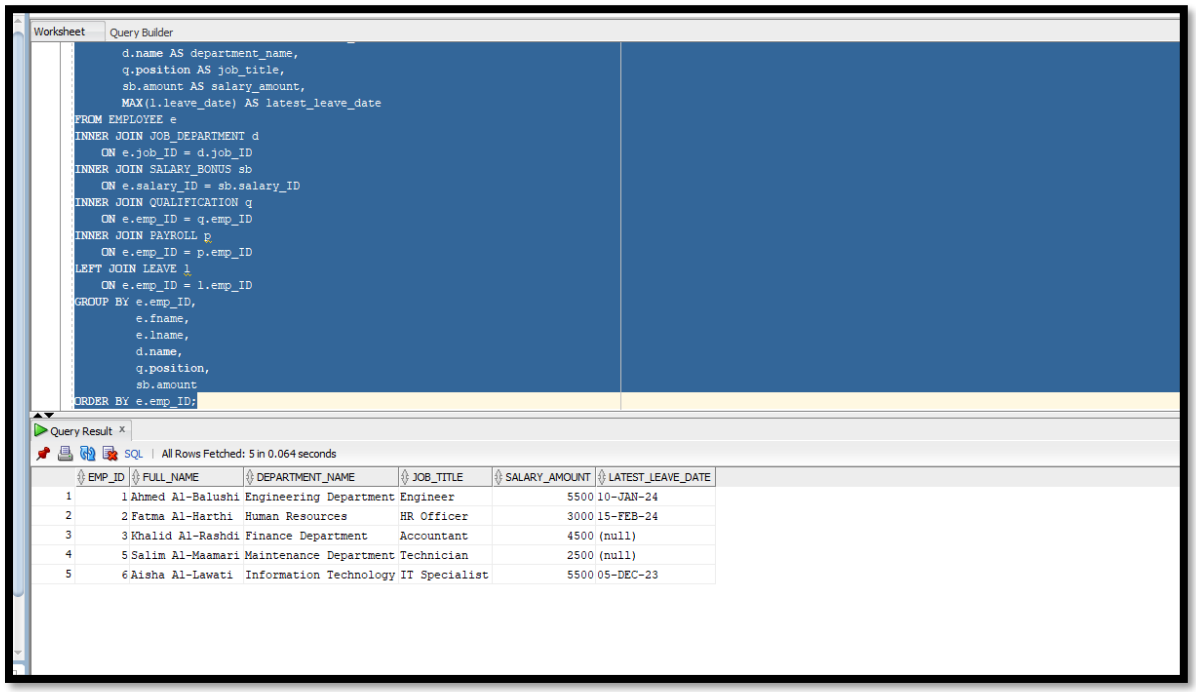
```sql
SELECT e.fname || ' ' || e.lname AS employee_name,
       d.name AS department_name,
       COUNT(l.leave_ID) AS leave_count
FROM EMPLOYEE e
JOIN JOB_DEPARTMENT d
    ON e.job_ID = d.job_ID
JOIN LEAVE l
    ON e.emp_ID = l.emp_ID
GROUP BY e.fname, e.lname, d.name
HAVING COUNT(l.leave_ID) > 1;
-- TASK 5 is Hard and is not included in this solution.
```

Script Output | Query Result

Task completed in 0.065 seconds

```
---------------------------------------------------- ------------
Maintenance Department                                         35
Finance Department                                           39.5

no rows selected
no rows selected
no rows selected
no rows selected
no rows selected
```

05_joins.sql — All join queries:



Worksheet | Query Builder

```
        d.name AS department_name,
        q.position AS job_title,
        sb.amount AS salary_amount,
        MAX(l.leave_date) AS latest_leave_date
FROM EMPLOYEE e
INNER JOIN JOB_DEPARTMENT d
    ON e.job_ID = d.job_ID
INNER JOIN SALARY_BONUS sb
    ON e.salary_ID = sb.salary_ID
INNER JOIN QUALIFICATION q
    ON e.emp_ID = q.emp_ID
INNER JOIN PAYROLL p
    ON e.emp_ID = p.emp_ID
LEFT JOIN LEAVE l
    ON e.emp_ID = l.emp_ID
GROUP BY e.emp_ID,
         e.fname,
         e.lname,
         d.name,
         q.position,
         sb.amount
ORDER BY e.emp_ID;
```

Query Result | SQL | All Rows Fetched: 5 in 0.064 seconds

	EMP_ID	FULL_NAME	DEPARTMENT_NAME	JOB_TITLE	SALARY_AMOUNT	LATEST_LEAVE_DATE
1	1	Ahmed Al-Balushi	Engineering Department	Engineer	5500	10-JAN-24
2	2	Fatma Al-Harthi	Human Resources	HR Officer	3000	15-FEB-24
3	3	Khalid Al-Rashdi	Finance Department	Accountant	4500	(null)
4	5	Salim Al-Maamari	Maintenance Department	Technician	2500	(null)
5	6	Aisha Al-Lawati	Information Technology	IT Specialist	5500	05-DEC-23

```sql
-- (a) List all employees who have never taken any leave.

SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS employee_name
FROM EMPLOYEE e
LEFT JOIN LEAVE l
    ON e.emp_ID = l.emp_ID
WHERE l.leave_ID IS NULL
ORDER BY e.emp_ID;


-- (b) List all departments that have no salary/bonus records.

SELECT d.job_ID,
       d.name AS department_name,
       d.job_dept
FROM JOB_DEPARTMENT d
LEFT JOIN SALARY_BONUS sb
    ON d.job_ID = sb.job_ID
WHERE sb.salary_ID IS NULL
ORDER BY d.job_ID;
```

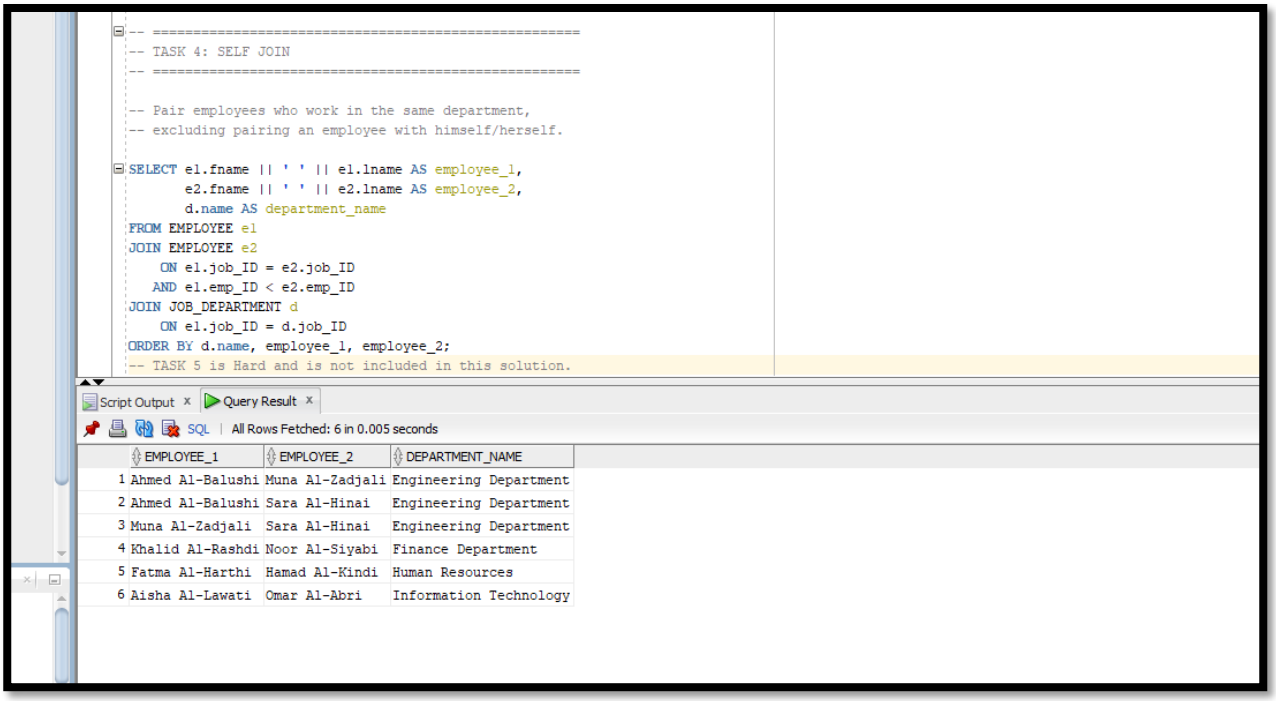
Query Result | Script Output | Task completed in 0.035 seconds

```
   EMP_ID EMPLOYEE_NAME
---------- ----------------------------------------------------------------------------------------------------
         3 Khalid Al-Rashdi
         5 Salim Al-Maamari
         7 Hamad Al-Kindi
         9 Omar Al-Abri
        10 Sara Al-Hinai

no rows selected
```

```sql
-- ============================================================
-- TASK 3: Multi-Table JOIN - Payroll Report
-- ============================================================

-- Complete payroll report joining all 6 tables.

SELECT p.payroll_ID,
       e.fname || ' ' || e.lname AS employee_name,
       d.name AS department_name,
       q.position AS position,
       sb.amount AS salary_amount,
       sb.bonus,
       l.reason AS leave_reason,
       p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e
    ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT d
    ON p.job_ID = d.job_ID
JOIN SALARY_BONUS sb
    ON p.salary_ID = sb.salary_ID
LEFT JOIN LEAVE l
    ON p.leave_ID = l.leave_ID
LEFT JOIN QUALIFICATION q
    ON e.emp_ID = q.emp_ID
ORDER BY d.name, p.total_amount DESC;
```

Script Output | Query Result | SQL | All Rows Fetched: 8 in 0.008 seconds

	PAYROLL_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	POSITION	SALARY_AMOUNT	BONUS	LEAVE_REASON	TOTAL_AMOUNT
1	1	Ahmed Al-Balushi	Engineering Department	Engineer	5500	700	Sick leave	5700
2	4	Muna Al-Zadjali	Engineering Department	(null)	5500	700	Emergency leave	5700
3	3	Khalid Al-Rashdi	Finance Department	Accountant	4500	600	(null)	5100
4	8	Noor Al-Siyabi	Finance Department	(null)	4500	600	Family reason	5100
5	2	Fatma Al-Harthi	Human Resources	HR Officer	3000	400	Annual leave	3400
6	7	Hamad Al-Kindi	Human Resources	(null)	3000	400	(null)	3400
7	6	Aisha Al-Lawati	Information Technology	IT Specialist	5500	800	Sick leave	6300
8	5	Salim Al-Maamari	Maintenance Department	Technician	2500	300	(null)	2800



```
-- ======================================================
-- TASK 4: SELF JOIN
-- ======================================================

-- Pair employees who work in the same department,
-- excluding pairing an employee with himself/herself.

SELECT e1.fname || ' ' || e1.lname AS employee_1,
       e2.fname || ' ' || e2.lname AS employee_2,
       d.name AS department_name
FROM EMPLOYEE e1
JOIN EMPLOYEE e2
    ON e1.job_ID = e2.job_ID
   AND e1.emp_ID < e2.emp_ID
JOIN JOB_DEPARTMENT d
    ON e1.job_ID = d.job_ID
ORDER BY d.name, employee_1, employee_2;
-- TASK 5 is Hard and is not included in this solution.
```

Script Output | Query Result

SQL | All Rows Fetched: 6 in 0.005 seconds

	EMPLOYEE_1	EMPLOYEE_2	DEPARTMENT_NAME
1	Ahmed Al-Balushi	Muna Al-Zadjali	Engineering Department
2	Ahmed Al-Balushi	Sara Al-Hinai	Engineering Department
3	Muna Al-Zadjali	Sara Al-Hinai	Engineering Department
4	Khalid Al-Rashdi	Noor Al-Siyabi	Finance Department
5	Fatma Al-Harthi	Hamad Al-Kindi	Human Resources
6	Aisha Al-Lawati	Omar Al-Abri	Information Technology

n 06_subqueries.sql — All subquery tasks:

```
    ON e.emp_ID = p.emp_ID
WHERE p.total_amount = (
    SELECT MAX(total_amount)
    FROM PAYROLL
);


-- (c) Find departments whose salary range belongs to
-- the department with the minimum salary amount.
```

Script Output

Task completed in 0.091 seconds

EMP_ID	EMPLOYEE_NAME	SALARY_AMOUNT
1	Ahmed Al-Balushi	5500
6	Aisha Al-Lawati	5500
9	Omar Al-Abri	5500
4	Muna Al-Zadjali	5500
10	Sara Al-Hinai	5500
8	Noor Al-Siyabi	4500
3	Khalid Al-Rashdi	4500

7 rows selected.

EMP_ID	EMPLOYEE_NAME	TOTAL_AMOUNT
6	Aisha Al-Lawati	6300

JOB_ID	DEPARTMENT_NAME	SALARY_RANGE
4	Maintenance Department	1000-3500

Script Output

Task completed in 0.076 seconds

JOB_ID	DEPARTMENT_NAME	SALARY_RANGE
4	Maintenance Department	1000-3500

EMP_ID	EMPLOYEE_NAME	JOB_ID
1	Ahmed Al-Balushi	1
2	Fatma Al-Harthi	2
3	Khalid Al-Rashdi	3
4	Muna Al-Zadjali	1
5	Salim Al-Maamari	4
6	Aisha Al-Lawati	5
7	Hamad Al-Kindi	2
8	Noor Al-Siyabi	3
9	Omar Al-Abri	5
10	Sara Al-Hinai	1

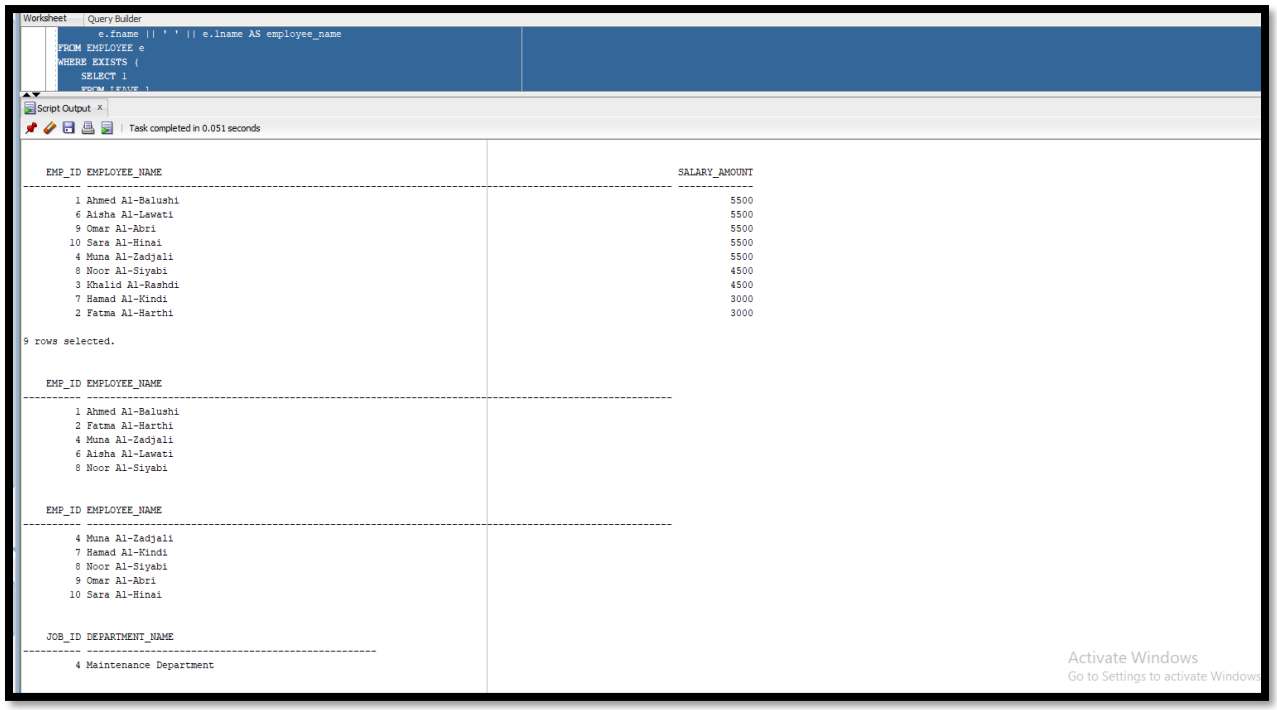
10 rows selected.

EMP_ID	EMPLOYEE_NAME	SALARY_AMOUNT
1	Ahmed Al-Balushi	5500
6	Aisha Al-Lawati	5500
9	Omar Al-Abri	5500
4	Muna Al-Zadjali	5500
10	Sara Al-Hinai	5500
8	Noor Al-Siyabi	4500
3	Khalid Al-Rashdi	4500

7 rows selected.

EMP_ID	EMPLOYEE_NAME	SALARY_AMOUNT
1	Ahmed Al-Balushi	5500
6	Aisha Al-Lawati	5500
9	Omar Al-Abri	5500
10	Sara Al-Hinai	5500
4	Muna Al-Zadjali	5500
8	Noor Al-Siyabi	4500
3	Khalid Al-Rashdi	4500

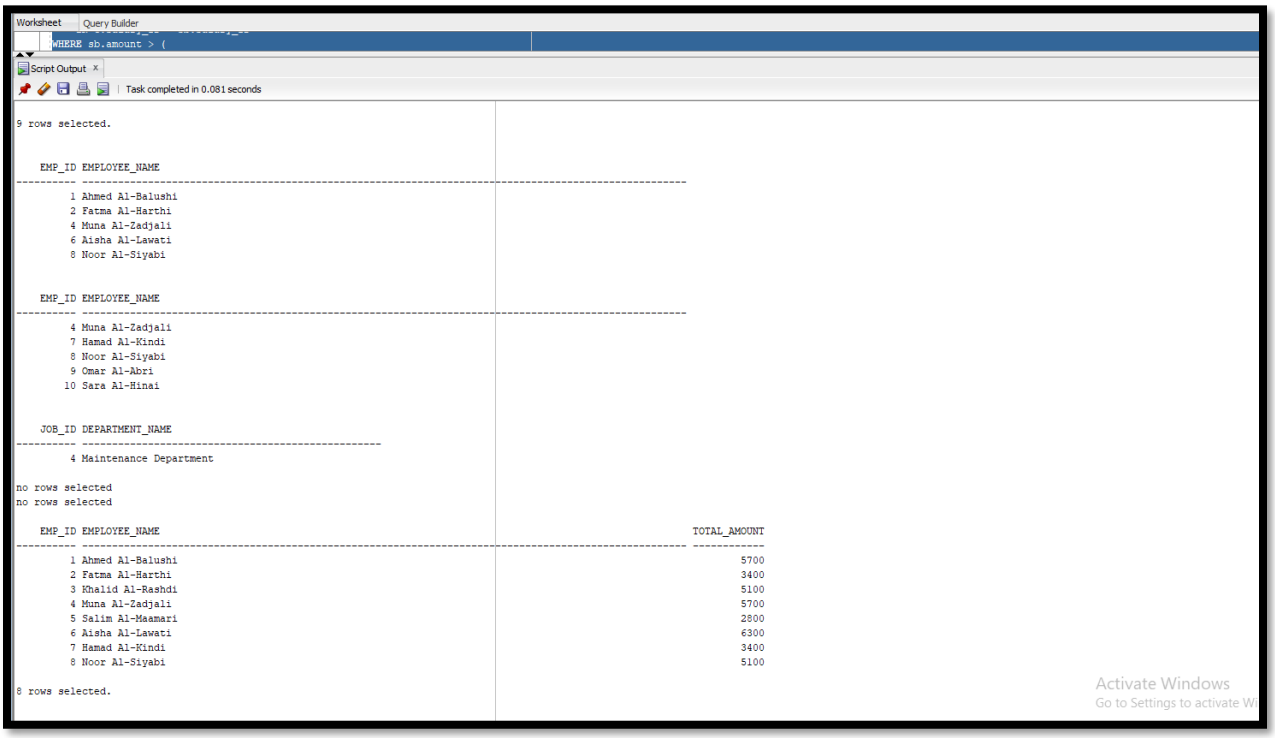
Task 2- Multi-Row Subqueries



```
Worksheet  Query Builder
        e.fname || ' ' || e.lname AS employee_name
FROM EMPLOYEE e
WHERE EXISTS (
    SELECT 1
    FROM [FAVT 1
```

```
Script Output
Task completed in 0.051 seconds

   EMP_ID EMPLOYEE_NAME                                                                                SALARY_AMOUNT
--------- ---------------------------------------------------------------------------------------------- -------------
        1 Ahmed Al-Balushi                                                                                        5500
        6 Aisha Al-Lawati                                                                                         5500
        9 Omar Al-Abri                                                                                            5500
       10 Sara Al-Hinai                                                                                           5500
        4 Muna Al-Zadjali                                                                                         5500
        8 Noor Al-Siyabi                                                                                          4500
        3 Khalid Al-Rashdi                                                                                        4500
        7 Hamad Al-Kindi                                                                                          3000
        2 Fatma Al-Harthi                                                                                         3000

9 rows selected.


   EMP_ID EMPLOYEE_NAME
--------- ----------------------------------------------------------------------------------------------
        1 Ahmed Al-Balushi
        2 Fatma Al-Harthi
        4 Muna Al-Zadjali
        6 Aisha Al-Lawati
        8 Noor Al-Siyabi


   EMP_ID EMPLOYEE_NAME
--------- ----------------------------------------------------------------------------------------------
        4 Muna Al-Zadjali
        7 Hamad Al-Kindi
        8 Noor Al-Siyabi
        9 Omar Al-Abri
       10 Sara Al-Hinai


   JOB_ID DEPARTMENT_NAME
--------- --------------------------------------------------
        4 Maintenance Department
```

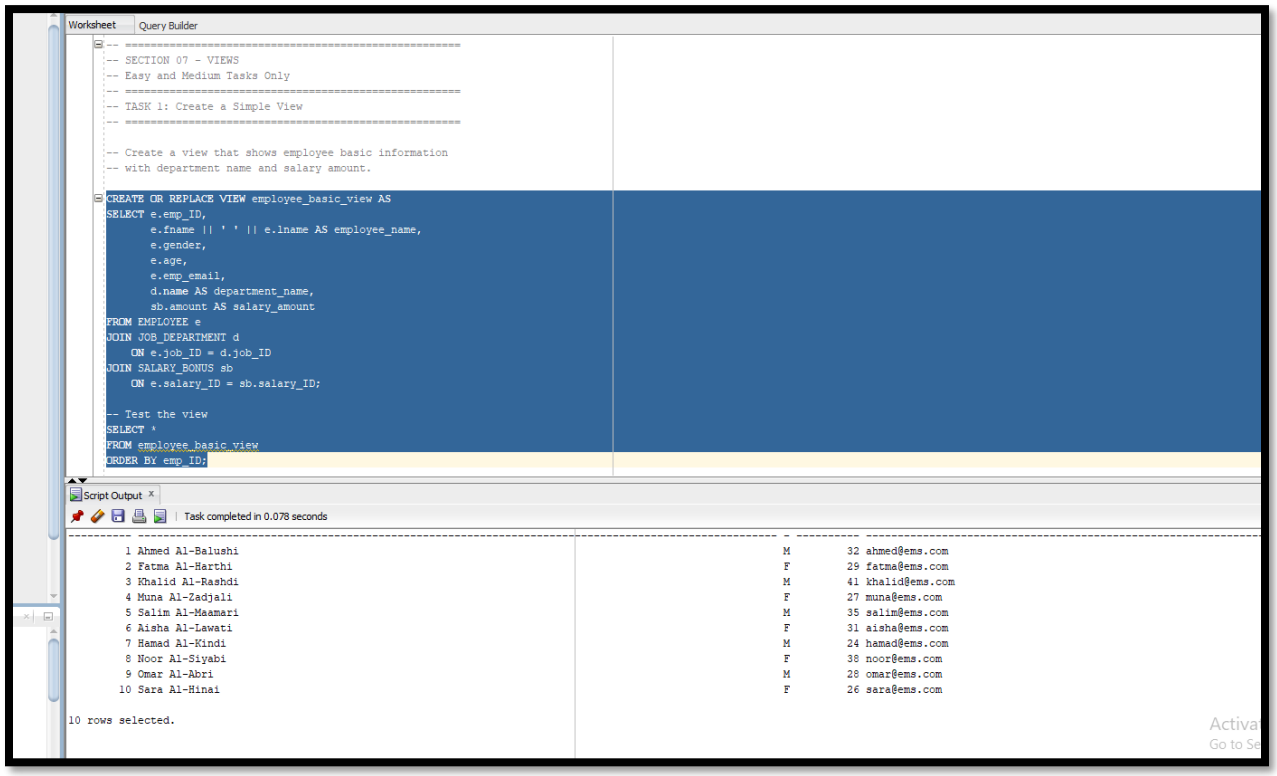

Task 3-EXISTS - NOT EXISTS



```
Worksheet  Query Builder
WHERE sb.amount > (
```

```
Script Output
Task completed in 0.081 seconds

9 rows selected.


   EMP_ID EMPLOYEE_NAME
--------- ----------------------------------------------------------------------------------------------
        1 Ahmed Al-Balushi
        2 Fatma Al-Harthi
        4 Muna Al-Zadjali
        6 Aisha Al-Lawati
        8 Noor Al-Siyabi


   EMP_ID EMPLOYEE_NAME
--------- ----------------------------------------------------------------------------------------------
        4 Muna Al-Zadjali
        7 Hamad Al-Kindi
        8 Noor Al-Siyabi
        9 Omar Al-Abri
       10 Sara Al-Hinai


   JOB_ID DEPARTMENT_NAME
--------- --------------------------------------------------
        4 Maintenance Department

no rows selected
no rows selected

   EMP_ID EMPLOYEE_NAME                                                                          TOTAL_AMOUNT
--------- ---------------------------------------------------------------------------------------- ------------
        1 Ahmed Al-Balushi                                                                                5700
        2 Fatma Al-Harthi                                                                                 3400
        3 Khalid Al-Rashdi                                                                                5100
        4 Muna Al-Zadjali                                                                                 5700
        5 Salim Al-Maamari                                                                                2800
        6 Aisha Al-Lawati                                                                                 6300
        7 Hamad Al-Kindi                                                                                  3400
        8 Noor Al-Siyabi                                                                                  5100

8 rows selected.
```

07_views.sql — All view definitions and test queries:



```
Worksheet  Query Builder

-- ============================================================
-- SECTION 07 - VIEWS
-- Easy and Medium Tasks Only
-- ============================================================
-- TASK 1: Create a Simple View
-- ============================================================

-- Create a view that shows employee basic information
-- with department name and salary amount.

CREATE OR REPLACE VIEW employee_basic_view AS
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS employee_name,
       e.gender,
       e.age,
       e.emp_email,
       d.name AS department_name,
       sb.amount AS salary_amount
FROM EMPLOYEE e
JOIN JOB_DEPARTMENT d
   ON e.job_ID = d.job_ID
JOIN SALARY_BONUS sb
   ON e.salary_ID = sb.salary_ID;

-- Test the view
SELECT *
FROM employee_basic_view
ORDER BY emp_ID;
```

Script Output — Task completed in 0.078 seconds

1	Ahmed Al-Balushi	M	32	ahmed@ems.com	
2	Fatma Al-Harthi	F	29	fatma@ems.com	
3	Khalid Al-Rashdi	M	41	khalid@ems.com	
4	Muna Al-Zadjali	F	27	muna@ems.com	
5	Salim Al-Maamari	M	35	salim@ems.com	
6	Aisha Al-Lawati	F	31	aisha@ems.com	
7	Hamad Al-Kindi	M	24	hamad@ems.com	
8	Noor Al-Siyabi	F	38	noor@ems.com	
9	Omar Al-Abri	M	28	omar@ems.com	
10	Sara Al-Hinai	F	26	sara@ems.com	

10 rows selected.

Section 07 - Task 1-Simple View

```
CREATE OR REPLACE VIEW department_payroll_summary AS
SELECT d.job_ID,
       d.name AS department_name,
       COUNT(DISTINCT e.emp_ID) AS total_employees,
       NVL(SUM(p.total_amount), 0) AS total_payroll,
       ROUND(AVG(p.total_amount), 2) AS average_payroll
FROM JOB_DEPARTMENT d
LEFT JOIN EMPLOYEE e
    ON d.job_ID = e.job_ID
LEFT JOIN PAYROLL p
    ON e.emp_ID = p.emp_ID
GROUP BY d.job_ID, d.name;
```

Script Output

Task completed in 0.09 seconds

```
         4 Muna Al-Zadjali                                                F         27 muna@ems.com
         5 Salim Al-Maamari                                               M         35 salim@ems.com
         6 Aisha Al-Lawati                                                F         31 aisha@ems.com
         7 Hamad Al-Kindi                                                 M         24 hamad@ems.com
         8 Noor Al-Siyabi                                                 F         38 noor@ems.com
         9 Omar Al-Abri                                                   M         28 omar@ems.com
        10 Sara Al-Hinai                                                  F         26 sara@ems.com

10 rows selected.


View DEPARTMENT_PAYROLL_SUMMARY created.


    JOB_ID DEPARTMENT_NAME                                TOTAL_EMPLOYEES TOTAL_PAYROLL AVERAGE_PAYROLL
---------- ---------------------------------------------- --------------- ------------- --------------
         1 Engineering Department                                       3         11400           5700
         3 Finance Department                                           2         10200           5100
         2 Human Resources                                              2          6800           3400
         5 Information Technology                                       2          6300           6300
         4 Maintenance Department                                       1          2800           2800
```

Task 2- Complex View

```
FROM EMPLOYEE;
```

Script Output

Task completed in 0.099 seconds

```
View EMPLOYEE_CONTACT_VIEW created.


    EMP_ID FNAME                                          LNAME                                          CONTACT_ADD
---------- ---------------------------------------------- ---------------------------------------------- ----------
         1 Ahmed                                          Al-Balushi                                     Muscat
         2 Fatma                                          Al-Harthi                                      Sohar
         3 Khalid                                         Al-Rashdi                                      Nizwa
         4 Muna                                           Al-Zadjali                                     Muscat
         5 Salim                                          Al-Maamari                                     Ibri
         6 Aisha                                          Al-Lawati                                      Seeb
         7 Hamad                                          Al-Kindi                                       Barka
         8 Noor                                           Al-Siyabi                                      Sur
         9 Omar                                           Al-Abri                                        Sohar
        10 Sara                                           Al-Hinai                                       Muscat

10 rows selected.


1 row updated.


    EMP_ID FNAME                                          LNAME                                          CONTACT_ADD
---------- ---------------------------------------------- ---------------------------------------------- ----------
         1 Ahmed                                          Al-Balushi                                     Muscat


1 row updated.


    EMP_ID FNAME                                          LNAME                                          CONTACT_ADD
---------- ---------------------------------------------- ---------------------------------------------- ----------
         1 Ahmed                                          Al-Balushi                                     Muscat


Commit complete.
```

Section 07 - Task 3- Updatable View

```
Worksheet  Query Builder
        fname,
        lname,
        gender,
        age,
        contact_add,
        emp_email,
        emp_pass,
        job_ID,
        salary_ID
FROM EMPLOYEE
WHERE job_ID = 1
WITH CHECK OPTION CONSTRAINT chk_engineering_view;


-- Test the view
SELECT emp_ID,
       fname || ' ' || lname AS employee_name,
       job_ID
FROM engineering_employees_view
ORDER BY emp_ID;


-- Valid update: update contact address while employee remains Engineering
UPDATE engineering_employees_view
```

Script Output | Task completed in 0.079 seconds

```
    EMP_ID EMPLOYEE_NAME                                                                                 JOB_ID
---------- ------------------------------------------------------------------------------------------ ----------
         1 Ahmed Al-Balushi                                                                                    1
         4 Muna Al-Zadjali                                                                                     1
        10 Sara Al-Hinai                                                                                       1


1 row updated.


    EMP_ID EMPLOYEE_NAME                                                                                 CONTACT_ADD
---------- ------------------------------------------------------------------------------------------ ----------------------------------------
         1 Ahmed Al-Balushi                                                                             Muscat - Updated


1 row updated.
```

Section 07 - Task 4- WITH CHECK OPTION

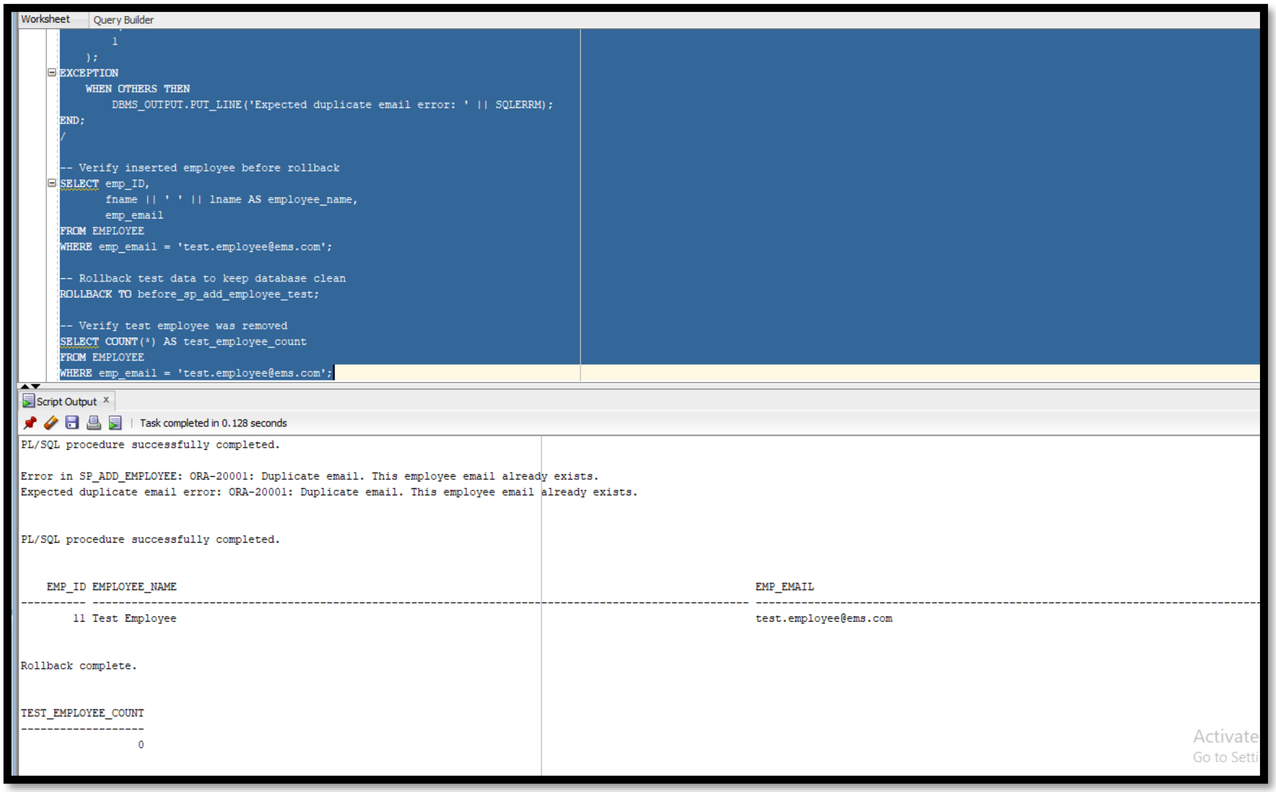
08_procedures.sql — All stored procedures, functions, and the package:

```
Worksheet  Query Builder
    )
    VALUES (
        employee_seq.NEXTVAL,
        p_fname,
        p_lname,
        p_gender,
        p_age,
        p_contact_add,
        LOWER(p_emp_email),
        p_emp_pass,
        p_job_ID,
        p_salary_ID
    );

    DBMS_OUTPUT.PUT_LINE('Employee added successfully: ' || p_fname || ' ' || p_lname);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in SP_ADD_EMPLOYEE: ' || SQLERRM);
        RAISE;
END;
/
```

Script Output | Task completed in 0.161 seconds

```
Procedure SP_ADD_EMPLOYEE compiled
```

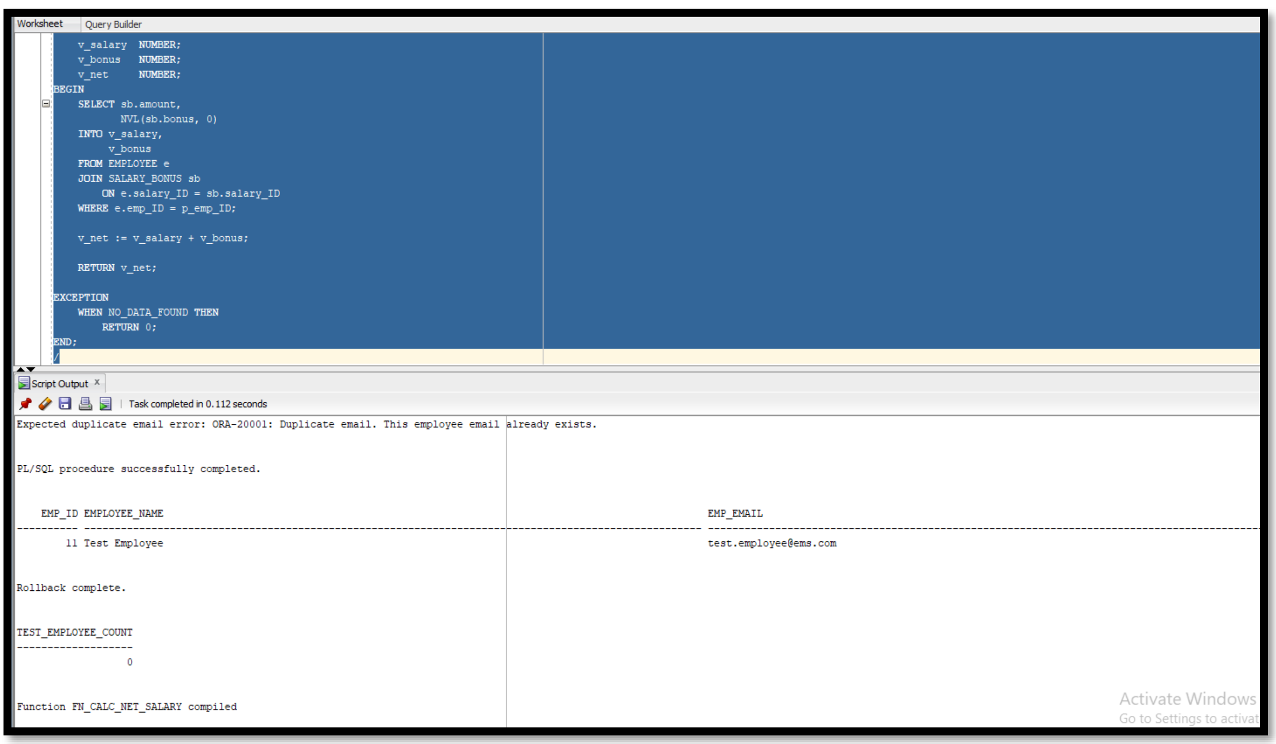
proceder1



```
        1
    );
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Expected duplicate email error: ' || SQLERRM);
END;
/

-- Verify inserted employee before rollback
SELECT emp_ID,
       fname || ' ' || lname AS employee_name,
       emp_email
FROM EMPLOYEE
WHERE emp_email = 'test.employee@ems.com';

-- Rollback test data to keep database clean
ROLLBACK TO before_sp_add_employee_test;

-- Verify test employee was removed
SELECT COUNT(*) AS test_employee_count
FROM EMPLOYEE
WHERE emp_email = 'test.employee@ems.com';
```

Script Output — Task completed in 0.128 seconds

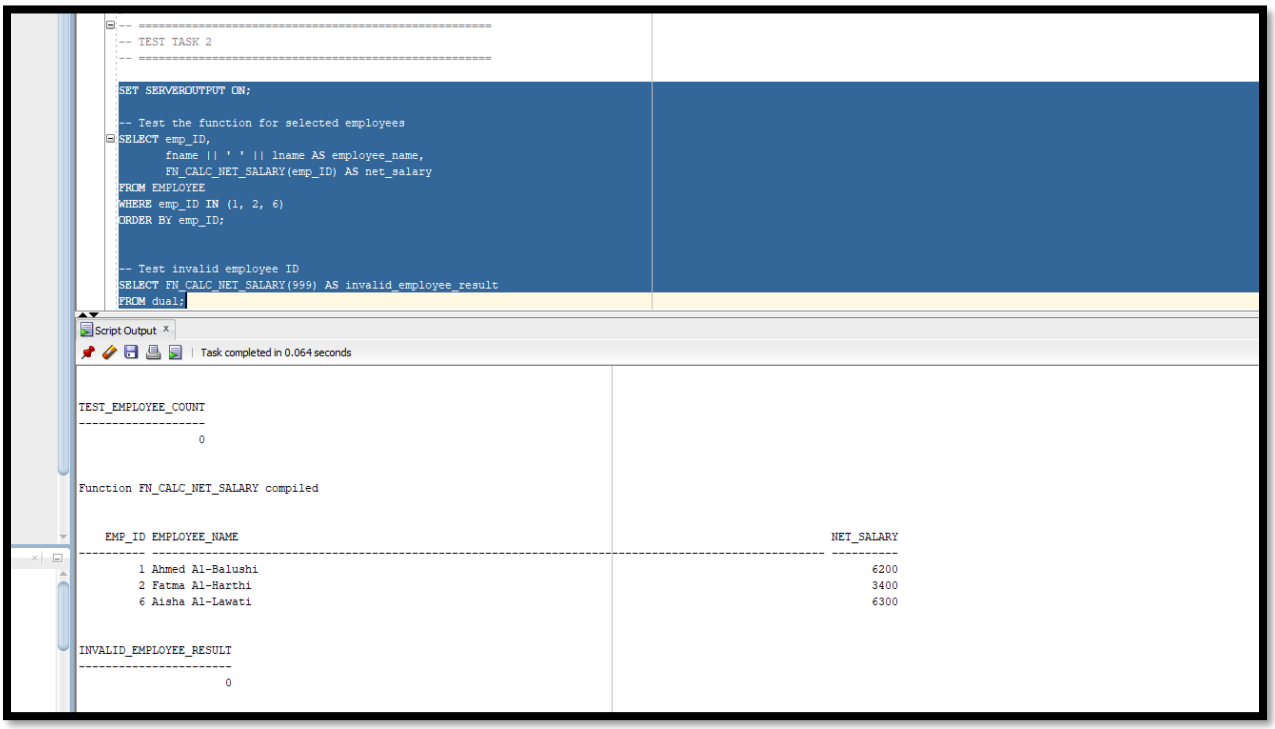
```
PL/SQL procedure successfully completed.

Error in SP_ADD_EMPLOYEE: ORA-20001: Duplicate email. This employee email already exists.
Expected duplicate email error: ORA-20001: Duplicate email. This employee email already exists.


PL/SQL procedure successfully completed.


    EMP_ID EMPLOYEE_NAME                                          EMP_EMAIL
---------- -----------------------------------------------------  ------------------------------------
        11 Test Employee                                          test.employee@ems.com


Rollback complete.


TEST_EMPLOYEE_COUNT
-------------------
                  0
```



```
    v_salary  NUMBER;
    v_bonus   NUMBER;
    v_net     NUMBER;
BEGIN
    SELECT sb.amount,
           NVL(sb.bonus, 0)
    INTO v_salary,
         v_bonus
    FROM EMPLOYEE e
    JOIN SALARY_BONUS sb
        ON e.salary_ID = sb.salary_ID
    WHERE e.emp_ID = p_emp_ID;

    v_net := v_salary + v_bonus;

    RETURN v_net;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END;
/
```

Script Output — Task completed in 0.112 seconds

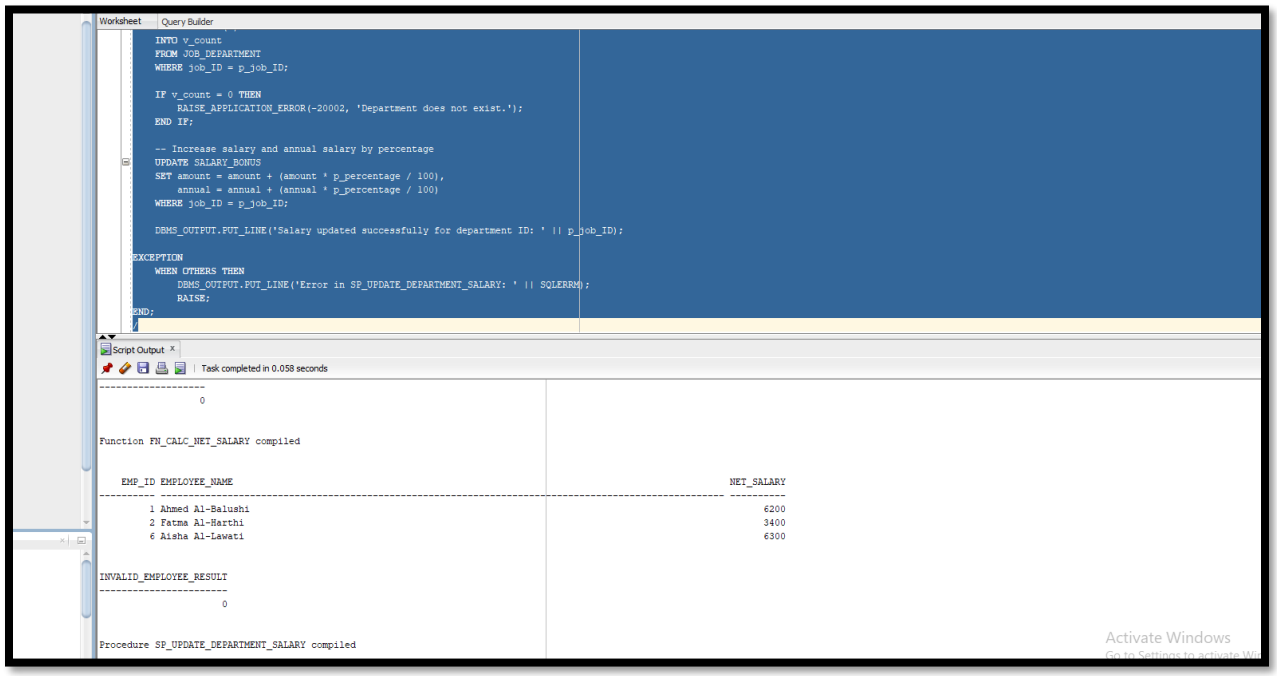
```
Expected duplicate email error: ORA-20001: Duplicate email. This employee email already exists.


PL/SQL procedure successfully completed.


    EMP_ID EMPLOYEE_NAME                                          EMP_EMAIL
---------- -----------------------------------------------------  ------------------------------------
        11 Test Employee                                          test.employee@ems.com


Rollback complete.


TEST_EMPLOYEE_COUNT
-------------------
                  0


Function FN_CALC_NET_SALARY compiled
```



```
-- ======================================================
-- TEST TASK 2
-- ======================================================

SET SERVEROUTPUT ON;

-- Test the function for selected employees
SELECT emp_ID,
       fname || ' ' || lname AS employee_name,
       FN_CALC_NET_SALARY(emp_ID) AS net_salary
FROM EMPLOYEE
WHERE emp_ID IN (1, 2, 6)
ORDER BY emp_ID;


-- Test invalid employee ID
SELECT FN_CALC_NET_SALARY(999) AS invalid_employee_result
FROM dual;
```

Script Output | Task completed in 0.064 seconds

```
TEST_EMPLOYEE_COUNT
-------------------
                  0


Function FN_CALC_NET_SALARY compiled


    EMP_ID EMPLOYEE_NAME                                                                                                 NET_SALARY
---------- ------------------------------------------------------------------------------------------------------------ ----------
         1 Ahmed Al-Balushi                                                                                                   6200
         2 Fatma Al-Harthi                                                                                                    3400
         6 Aisha Al-Lawati                                                                                                    6300


INVALID_EMPLOYEE_RESULT
-----------------------
                      0
```

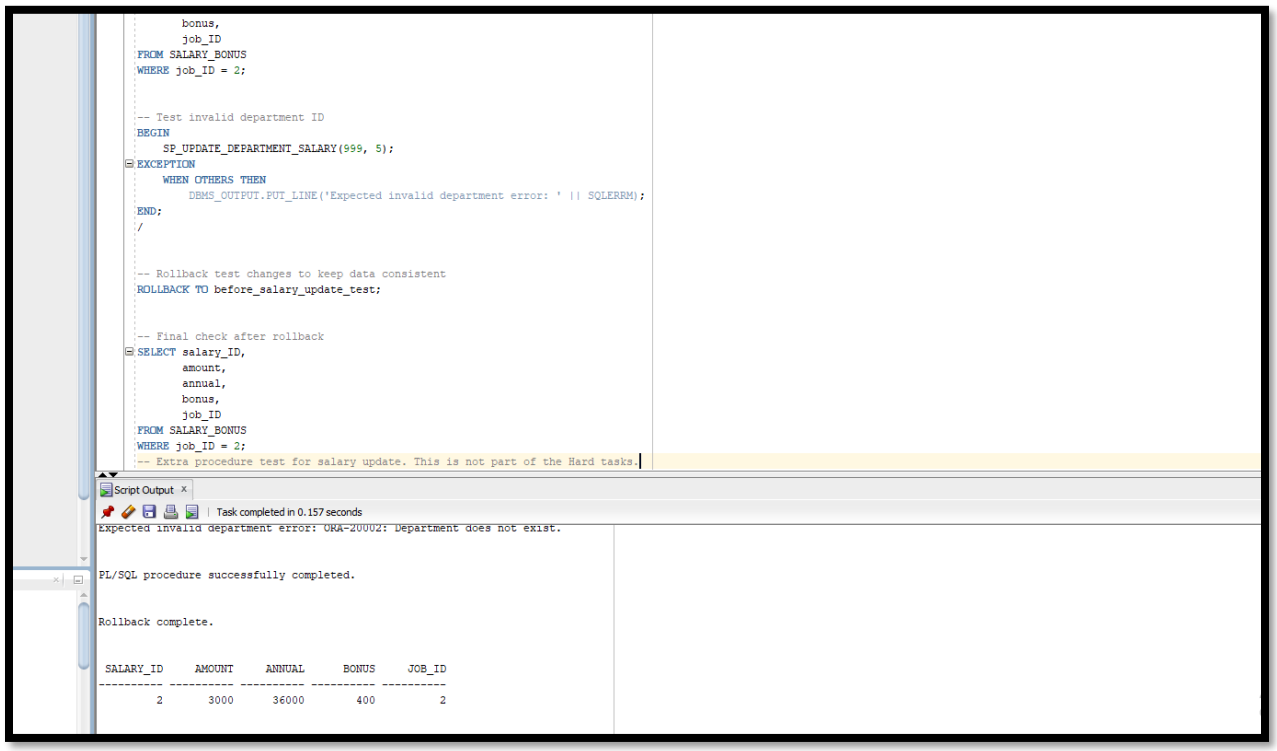


```
    INTO v_count
    FROM JOB_DEPARTMENT
    WHERE job_ID = p_job_ID;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Department does not exist.');
    END IF;

    -- Increase salary and annual salary by percentage
    UPDATE SALARY_BONUS
    SET amount = amount + (amount * p_percentage / 100),
        annual = annual + (annual * p_percentage / 100)
    WHERE job_ID = p_job_ID;

    DBMS_OUTPUT.PUT_LINE('Salary updated successfully for department ID: ' || p_job_ID);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in SP_UPDATE_DEPARTMENT_SALARY: ' || SQLERRM);
        RAISE;
END;
/
```

Script Output | Task completed in 0.058 seconds

```
-------------------
                  0


Function FN_CALC_NET_SALARY compiled


    EMP_ID EMPLOYEE_NAME                                                                                                 NET_SALARY
---------- ------------------------------------------------------------------------------------------------------------ ----------
         1 Ahmed Al-Balushi                                                                                                   6200
         2 Fatma Al-Harthi                                                                                                    3400
         6 Aisha Al-Lawati                                                                                                    6300


INVALID_EMPLOYEE_RESULT
-----------------------
                      0


Procedure SP_UPDATE_DEPARTMENT_SALARY compiled
```

Task 3- Procedure Update Salary



```
        bonus,
        job_ID
FROM SALARY_BONUS
WHERE job_ID = 2;


-- Test invalid department ID
BEGIN
    SP_UPDATE_DEPARTMENT_SALARY(999, 5);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Expected invalid department error: ' || SQLERRM);
END;
/


-- Rollback test changes to keep data consistent
ROLLBACK TO before_salary_update_test;


-- Final check after rollback
SELECT salary_ID,
       amount,
       annual,
       bonus,
       job_ID
FROM SALARY_BONUS
WHERE job_ID = 2;
-- Extra procedure test for salary update. This is not part of the Hard tasks.
```

Script Output

Task completed in 0.157 seconds

```
Expected invalid department error: ORA-20002: Department does not exist.


PL/SQL procedure successfully completed.


Rollback complete.


 SALARY_ID     AMOUNT     ANNUAL      BONUS     JOB_ID
---------- ---------- ---------- ---------- ----------
         2       3000      36000        400          2
```

```
            salary_ID,
            leave_ID
        )
        VALUES (
            payroll_seq.NEXTVAL,
            p_pay_date,
            'Monthly payroll processed by procedure',
            ROUND(v_total_amount, 2),
            v_emp_rec.emp_ID,
            v_emp_rec.job_ID,
            v_emp_rec.salary_ID,
            NULL
        );

        v_insert_count := v_insert_count + 1;
    END LOOP;

    CLOSE emp_cur;

    DBMS_OUTPUT.PUT_LINE('Payroll records inserted: ' || v_insert_count);

EXCEPTION
    WHEN OTHERS THEN
        IF emp_cur%ISOPEN THEN
            CLOSE emp_cur;
        END IF;

        DBMS_OUTPUT.PUT_LINE('Error in SP_PROCESS_PAYROLL: ' || SQLERRM);
        RAISE;
END;
/
```

```
Task completed in 0.089 seconds

PL/SQL procedure successfully completed.


Rollback complete.


 SALARY_ID     AMOUNT     ANNUAL      BONUS     JOB_ID
---------- ---------- ---------- ---------- ----------
         2       3000      36000        400          2


Procedure SP_PROCESS_PAYROLL compiled
```

procedure payroll create

```
SELECT COUNT(*) AS payroll_count_before
FROM PAYROLL;

-- Run payroll process for Engineering department on a test date
BEGIN
    SP_PROCESS_PAYROLL(1, DATE '2024-05-31');
END;
/

-- Show inserted test payroll records
SELECT payroll_ID,
       payroll_date,
       report,
       total_amount,
       emp_ID,
       job_ID,
       salary_ID
FROM PAYROLL
WHERE payroll_date = DATE '2024-05-31'
ORDER BY emp_ID;

-- Count payroll records after running the procedure
SELECT COUNT(*) AS payroll_count_after
FROM PAYROLL;

-- Rollback test records to keep original data clean
ROLLBACK TO before_process_payroll_test;

-- Final count should return to original number
SELECT COUNT(*) AS payroll_count_final
```

```
Task completed in 0.133 seconds

PAYROLL_COUNT_AFTER
-------------------
                 11


Rollback complete.


PAYROLL_COUNT_FINAL
-------------------
                  8
```

```
    )
    SELECT salary_audit_seq.NEXTVAL,
           salary_ID,
           job_ID,
           amount AS old_amount,
           amount + (amount * p_percentage / 100) AS new_amount,
           annual AS old_annual,
           annual + (annual * p_percentage / 100) AS new_annual,
           p_percentage,
           SYSDATE,
           USER
    FROM SALARY_BONUS
    WHERE job_ID = p_job_ID;

    -- Update salary
    UPDATE SALARY_BONUS
    SET amount = amount + (amount * p_percentage / 100),
        annual = annual + (annual * p_percentage / 100)
    WHERE job_ID = p_job_ID;
```

Script Output | Task completed in 0.108 seconds

```
Rollback complete.


PAYROLL_COUNT_FINAL
-------------------
                  8

SALARY_AUDIT table created.


PL/SQL procedure successfully completed.

SALARY_AUDIT_SEQ created.


PL/SQL procedure successfully completed.


Procedure SP_RAISE_SALARY compiled
```


Task 4-Salary Raise with Audit Table

```
BEGIN
    SP_RAISE_SALARY(3, -5);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Expected invalid percentage error: ' || SQLERRM);
END;
/


-- Rollback test changes to keep original data clean
ROLLBACK TO before_raise_salary_test;


-- Final check after rollback
SELECT salary_ID,
       job_ID,
       amount,
       annual,
```

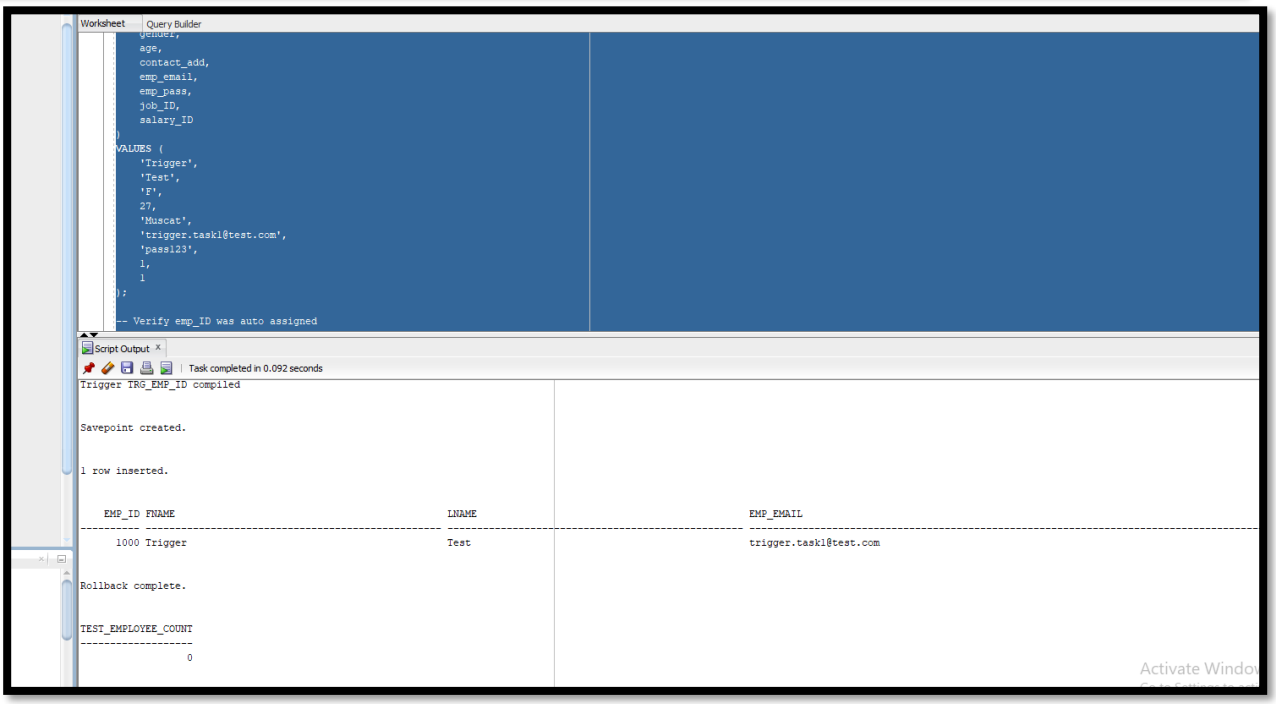
Script Output | Task completed in 0.175 seconds

```
 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4950      59400        600


  AUDIT_ID  SALARY_ID     JOB_ID OLD_AMOUNT NEW_AMOUNT OLD_ANNUAL NEW_ANNUAL RAISE_PERCENTAGE CHANGED_BY
---------- ---------- ---------- ---------- ---------- ---------- ---------- ---------------- ----------------------------------------------------
         1          3          3       4500       4950      54000      59400               10 KAWTHER_EMS

Error in SP_RAISE_SALARY: ORA-20004: Raise percentage must be greater than zero.
Expected invalid percentage error: ORA-20004: Raise percentage must be greater than zero.


PL/SQL procedure successfully completed.


Rollback complete.


 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600
```

09_triggers.sql — All trigger definitions + screenshots:

```
-- ================================================
-- SECTION 09 - TRIGGERS
-- Easy and Medium Tasks Only
-- ================================================
-- ================================================
-- TASK 1: BEFORE INSERT - Auto Assign emp_ID
-- ================================================

SET SERVEROUTPUT ON;

-- Create EMP_SEQ if it does not already exist
BEGIN
    EXECUTE IMMEDIATE '
        CREATE SEQUENCE EMP_SEQ
        START WITH 1000
        INCREMENT BY 1
    ';

    DBMS_OUTPUT.PUT_LINE('EMP_SEQ created.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -955 THEN
            DBMS_OUTPUT.PUT_LINE('EMP_SEQ already exists.');
        ELSE
            RAISE;
        END IF;
END;
/

-- Create trigger to auto assign emp_ID
CREATE OR REPLACE TRIGGER TRG_EMP_ID
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    IF :NEW.emp_ID IS NULL THEN
        :NEW.emp_ID := EMP_SEQ.NEXTVAL;
```

```
Script Output | Task completed in 0.127 seconds

PL/SQL procedure successfully completed.

Trigger TRG_EMP_ID compiled
```



```
Worksheet | Query Builder
        gender,
        age,
        contact_add,
        emp_email,
        emp_pass,
        job_ID,
        salary_ID
    )
    VALUES (
        'Trigger',
        'Test',
        'F',
        27,
        'Muscat',
        'trigger.task1@test.com',
        'pass123',
        1,
        1
    );

    -- Verify emp_ID was auto assigned
```

```
Script Output | Task completed in 0.092 seconds

Trigger TRG_EMP_ID compiled


Savepoint created.


1 row inserted.


    EMP_ID FNAME                                              LNAME                                             EMP_EMAIL
---------- -------------------------------------------------- ------------------------------------------------- --------------------------------------------------
      1000 Trigger                                            Test                                              trigger.task1@test.com


Rollback complete.


TEST_EMPLOYEE_COUNT
-------------------
                  0
```

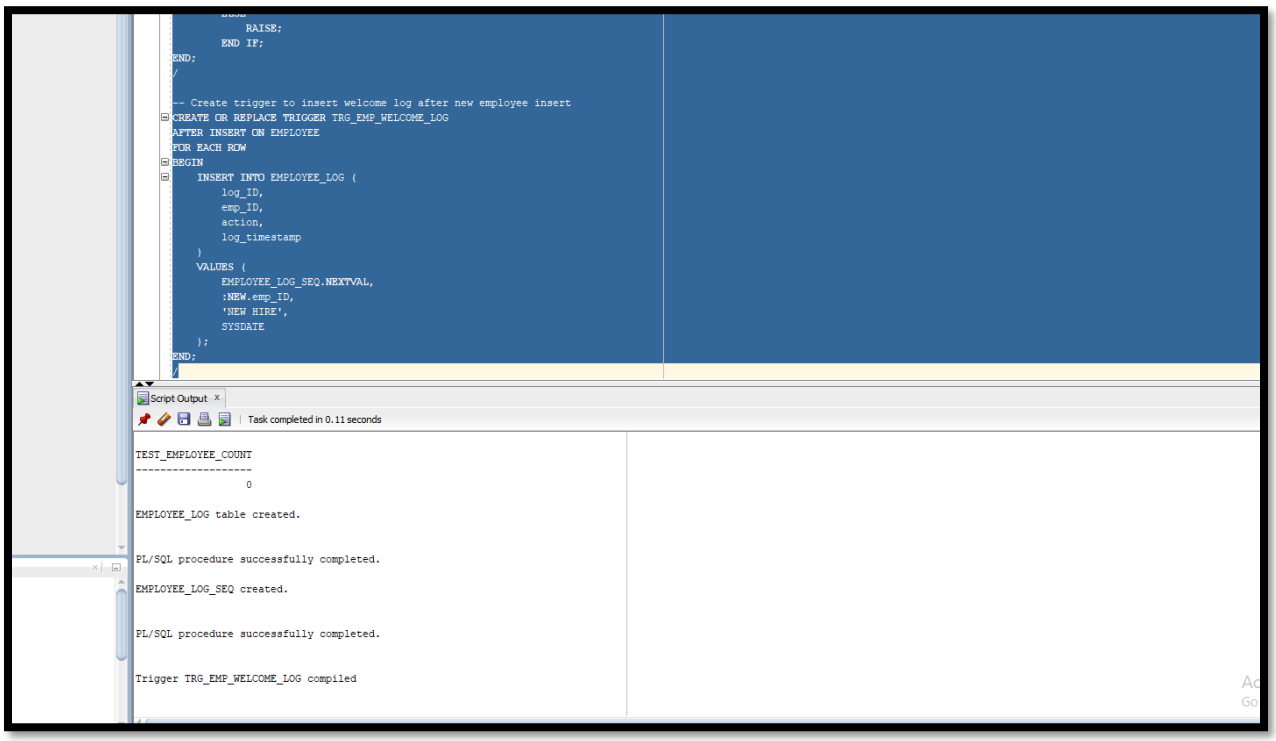
task1-emp_ID = 1000

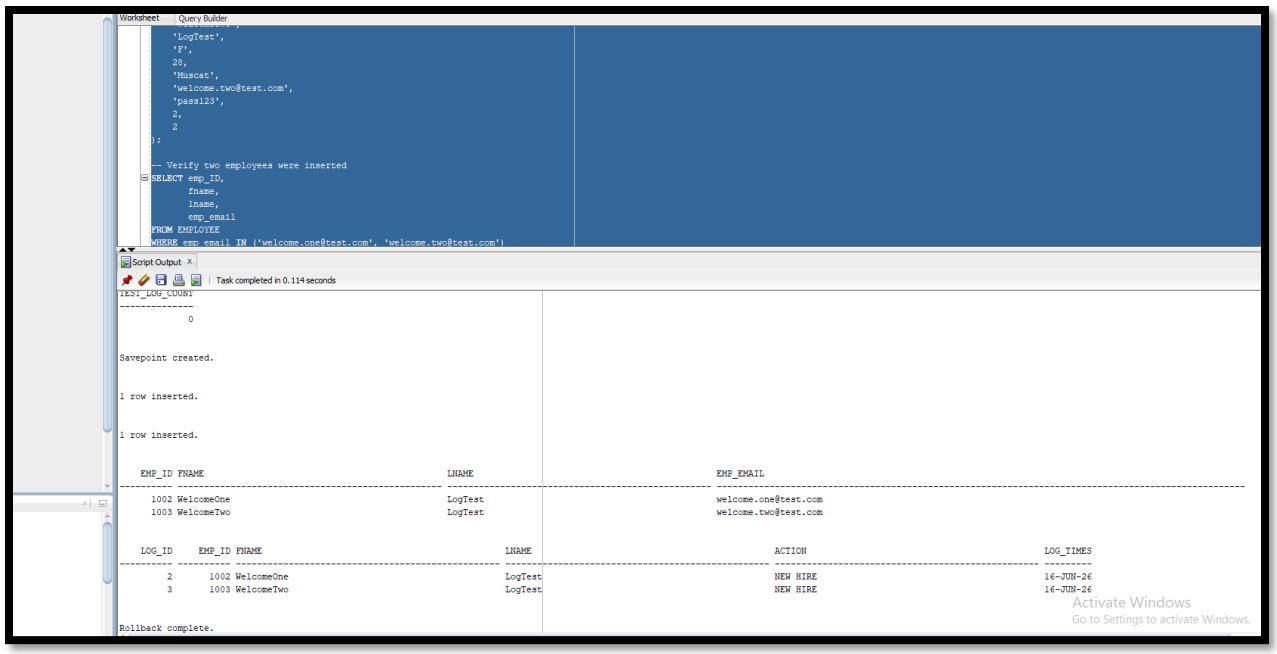
```
-- Verify log was inserted automatically
SELECT l.log_ID,
       l.emp_ID,
       l.action,
       l.log_timestamp
FROM EMPLOYEE_LOG l
JOIN EMPLOYEE e
    ON l.emp_ID = e.emp_ID
WHERE e.emp_email = 'welcome.logtest@test.com';

-- Rollback test data
ROLLBACK TO before_trg_welcome_log_test;

-- Final check: employee should be removed
SELECT COUNT(*) AS test_employee_count
FROM EMPLOYEE
```

Script Output

Task completed in 0.142 seconds

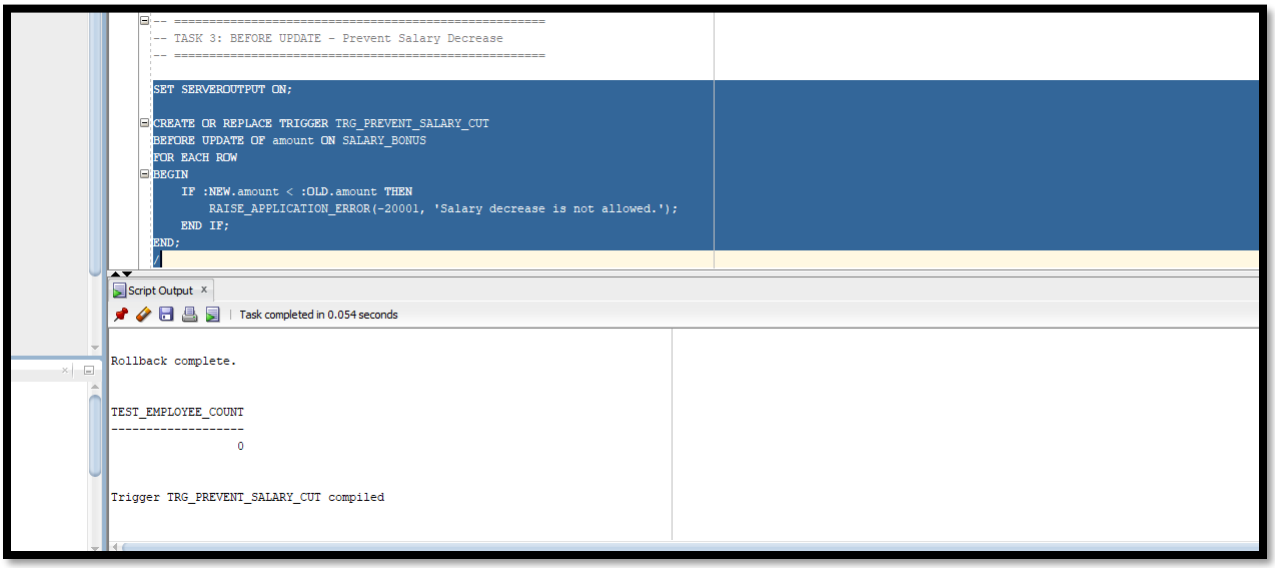
```
1 row inserted.


    EMP_ID FNAME                                              LNAME                                             EMP_EMAIL
---------- -------------------------------------------------- ------------------------------------------------- -------------------------------------------------
      1001 Welcome                                            LogTest                                           welcome.logtest@test.com


    LOG_ID     EMP_ID ACTION                                             LOG_TIMES
---------- ---------- -------------------------------------------------- ---------
         1       1001 NEW HIRE                                           16-JUN-26


Rollback complete.


TEST_EMPLOYEE_COUNT
-------------------
                  0


TEST_LOG_COUNT
--------------
```


RAISE;
END IF;
END;
/
-- Create trigger to insert welcome log after new employee insert
CREATE OR REPLACE TRIGGER TRG_EMP_WELCOME_LOG
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
INSERT INTO EMPLOYEE_LOG (
log_ID,
emp_ID,
action,
log_timestamp
)
VALUES (
EMPLOYEE_LOG_SEQ.NEXTVAL,
:NEW.emp_ID,
'NEW HIRE',
SYSDATE
);
END;
/
Script Output
Task completed in 0.11 seconds
TEST_EMPLOYEE_COUNT
0
EMPLOYEE_LOG table created.
PL/SQL procedure successfully completed.
EMPLOYEE_LOG_SEQ created.
PL/SQL procedure successfully completed.
Trigger TRG_EMP_WELCOME_LOG compiled

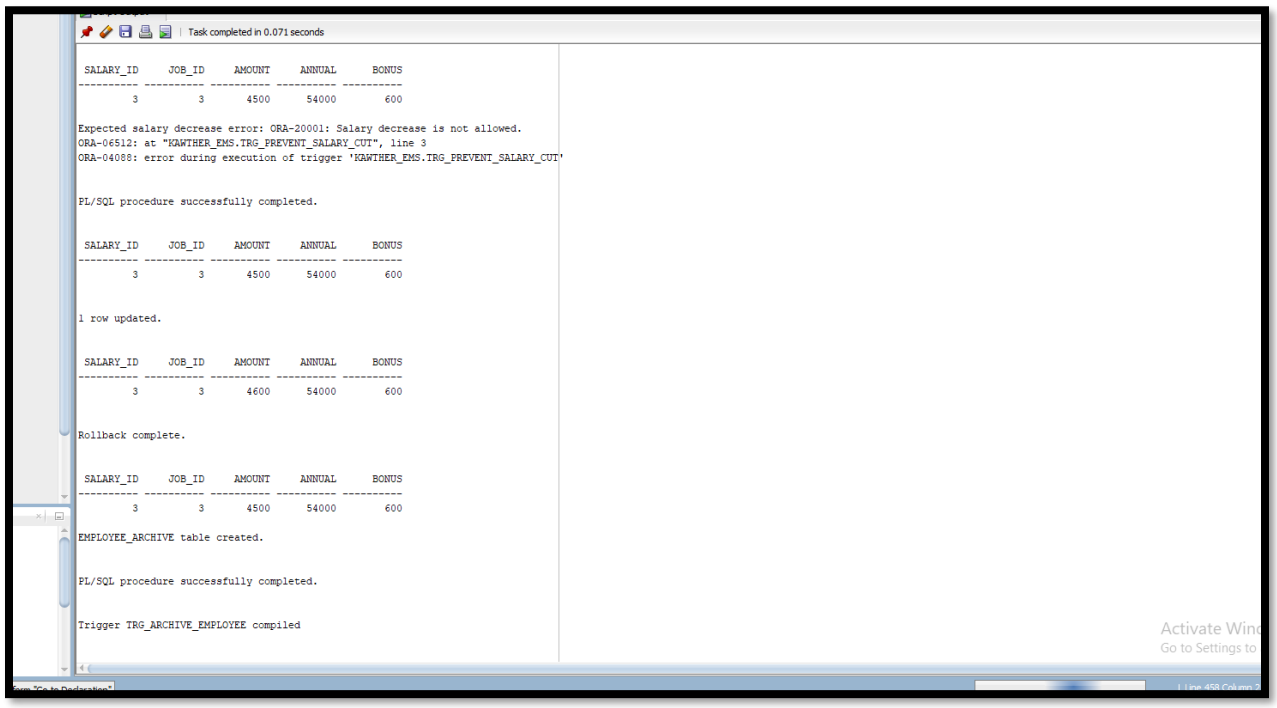


---

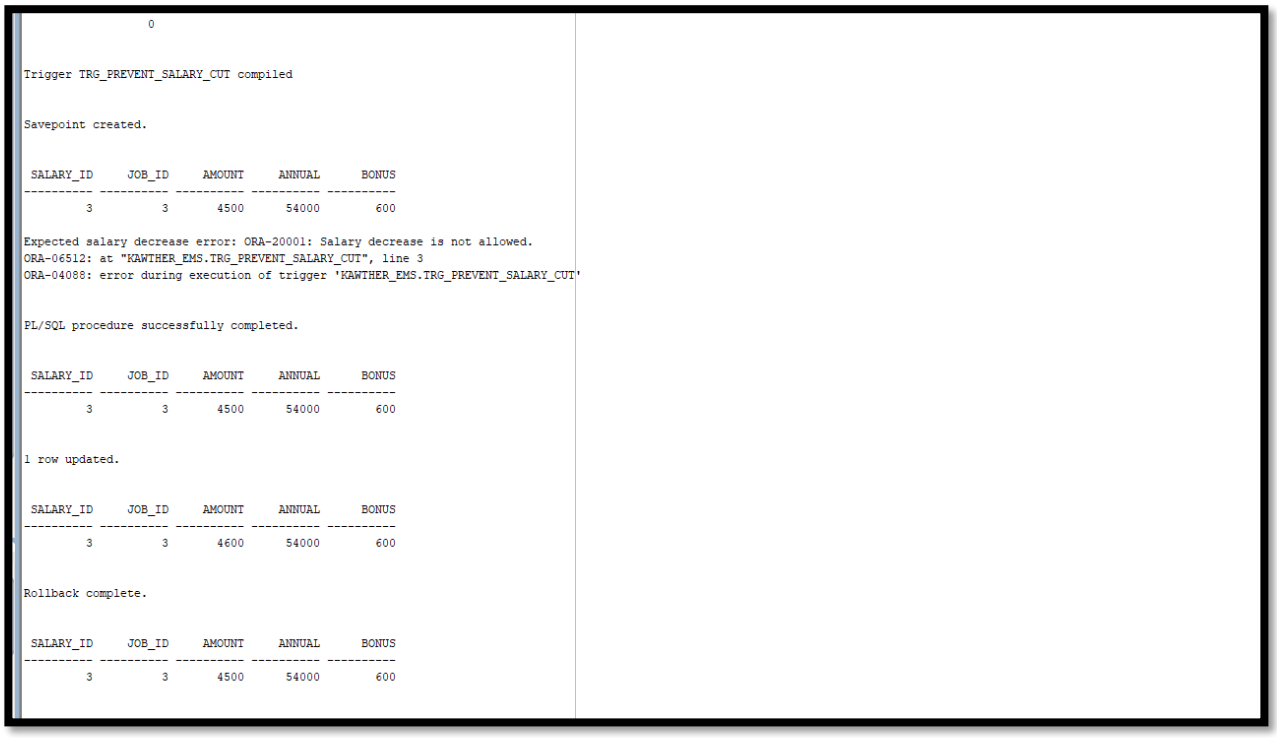
employee2-s9-task9



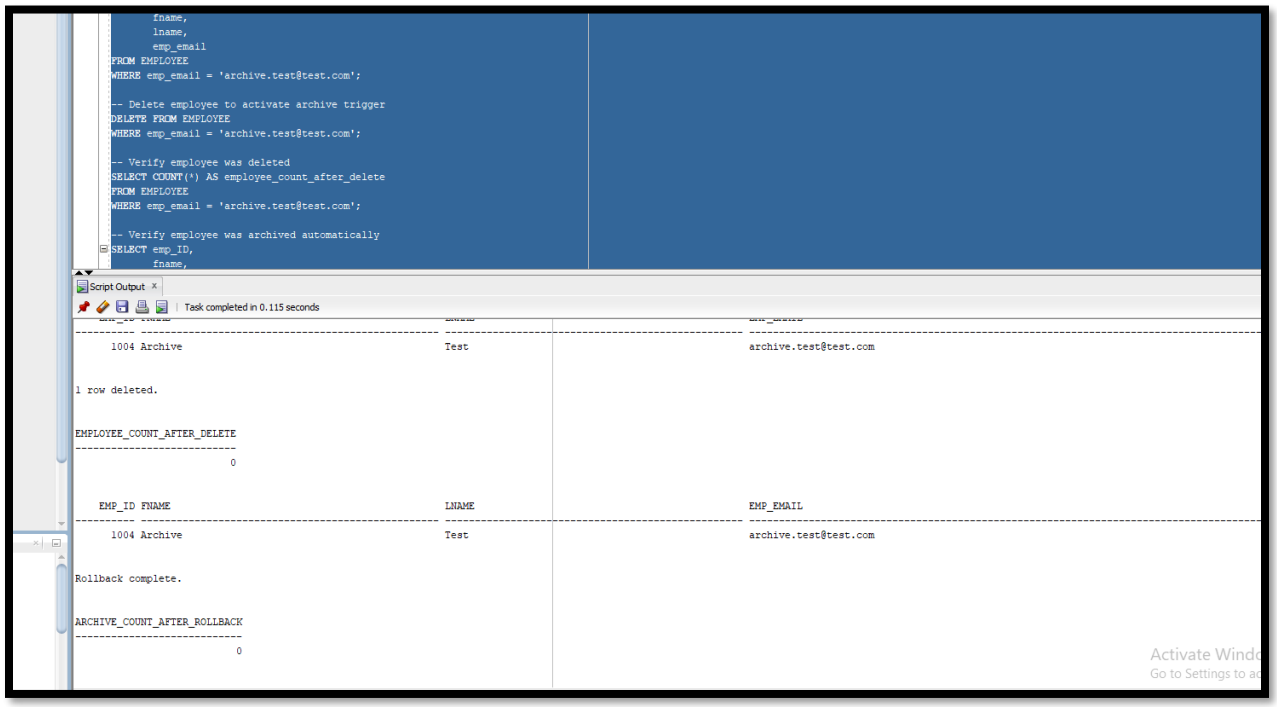
TASK 3-s9- BEFORE UPDATE - Prevent Salary Decrease



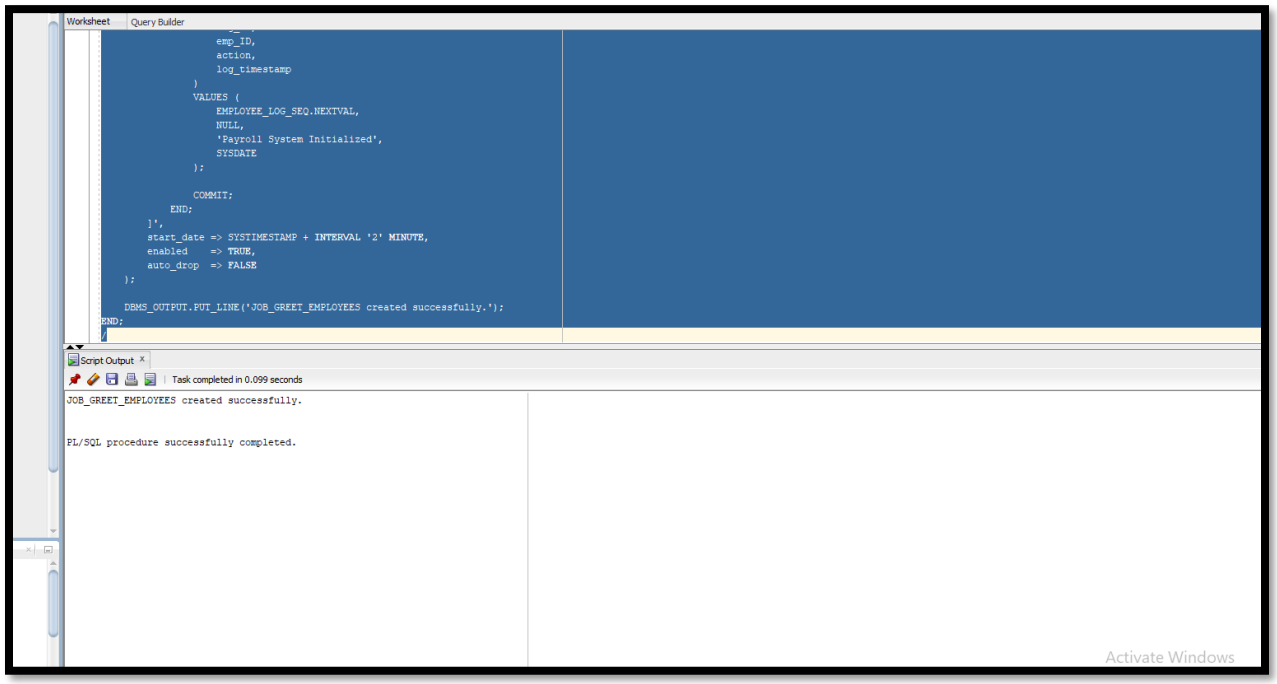
```
Task completed in 0.071 seconds

 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600

Expected salary decrease error: ORA-20001: Salary decrease is not allowed.
ORA-06512: at "KAWTHER_EMS.TRG_PREVENT_SALARY_CUT", line 3
ORA-04088: error during execution of trigger 'KAWTHER_EMS.TRG_PREVENT_SALARY_CUT'


PL/SQL procedure successfully completed.

 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600


1 row updated.

 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4600      54000        600


Rollback complete.

 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600

EMPLOYEE_ARCHIVE table created.


PL/SQL procedure successfully completed.


Trigger TRG_ARCHIVE_EMPLOYEE compiled
```

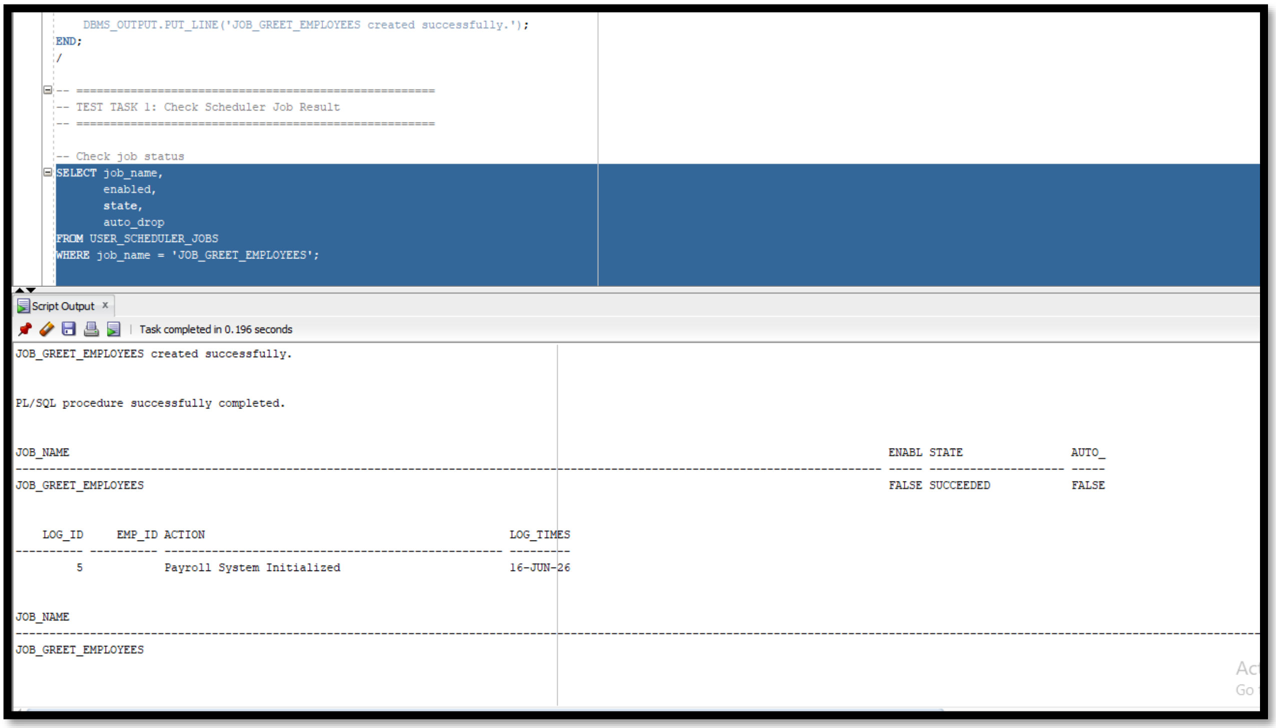


```
                  0


Trigger TRG_PREVENT_SALARY_CUT compiled


Savepoint created.


 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600

Expected salary decrease error: ORA-20001: Salary decrease is not allowed.
ORA-06512: at "KAWTHER_EMS.TRG_PREVENT_SALARY_CUT", line 3
ORA-04088: error during execution of trigger 'KAWTHER_EMS.TRG_PREVENT_SALARY_CUT'


PL/SQL procedure successfully completed.


 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600


1 row updated.


 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4600      54000        600


Rollback complete.


 SALARY_ID     JOB_ID     AMOUNT     ANNUAL      BONUS
---------- ---------- ---------- ---------- ----------
         3          3       4500      54000        600
```



n 10_scheduler.sql — All scheduler job scripts + screenshots:





Test after 2 minute

```
                )
                VALUES (
                    EMPLOYEE_LOG_SEQ.NEXTVAL,
                    NULL,
                    'Daily Leave Count: ' || v_leave_count,
                    SYSDATE
                );

                COMMIT;
            END;
        ]',
        start_date      => SYSTIMESTAMP,
        repeat_interval => 'FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0',
        enabled         => TRUE,
        auto_drop       => FALSE
    );

    DBMS_OUTPUT.PUT_LINE('JOB_DAILY_LEAVE_REPORT created successfully.');
END;
/
```

Script Output — Task completed in 0.048 seconds

```
JOB_NAME                                      ENABL STATE                AUTO_
--------------------------------------------- ----- -------------------- -----
JOB_GREET_EMPLOYEES                           FALSE SUCCEEDED            FALSE


    LOG_ID     EMP_ID ACTION                                             LOG_TIMES
---------- ---------- -------------------------------------------------- ---------
         5            Payroll System Initialized                         16-JUN-26


JOB_NAME
--------------------------------------------------------------------------------
JOB_GREET_EMPLOYEES

JOB_DAILY_LEAVE_REPORT created successfully.


PL/SQL procedure successfully completed.
```

- daily scheduler job-s10-t2

```
-- ====================================================
-- TEST TASK 2: Check Daily Leave Report Job
-- ====================================================

SELECT job_name,
       enabled,
       state,
       repeat_interval,
       auto_drop
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_DAILY_LEAVE_REPORT';
```

Script Output — Task completed in 0.126 seconds

```
JOB_NAME
--------------------------------------------------------------------------------
JOB_GREET_EMPLOYEES

JOB_DAILY_LEAVE_REPORT created successfully.


PL/SQL procedure successfully completed.


JOB_NAME                                                         ENABL STATE
---------------------------------------------------------------- ----- --------------------
REPEAT_INTERVAL
--------------------------------------------------------------------------------
AUTO_
-----
JOB_DAILY_LEAVE_REPORT                                           TRUE  SCHEDULED
FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0
FALSE
```


task2.

```
    );

    DBMS_SCHEDULER.ENABLE('JOB_MONTHLY_PAYROLL');

    DBMS_OUTPUT.PUT_LINE('PROG_MONTHLY_PAYROLL created successfully.');
    DBMS_OUTPUT.PUT_LINE('SCH_FIRST_OF_MONTH created successfully.');
    DBMS_OUTPUT.PUT_LINE('JOB_MONTHLY_PAYROLL created successfully.');
END;
/
```

Script Output

Task completed in 0.274 seconds

```
NUMBER_OF_ARGUMENTS ENABL
------------------- -----
PROG_MONTHLY_PAYROLL                                                STORED_PROCEDURE
SP_PROCESS_PAYROLL
                  2 TRUE



SCHEDULE_NAME
--------------------------------------------------------------------------------
REPEAT_INTERVAL
--------------------------------------------------------------------------------
SCH_FIRST_OF_MONTH
FREQ=MONTHLY; BYMONTHDAY=1; BYHOUR=6; BYMINUTE=0; BYSECOND=0



JOB_NAME                                                            ENABL STATE
------------------------------------------------------------------------------- ----- --------------------
PROGRAM_NAME
--------------------------------------------------------------------------------
SCHEDULE_NAME
--------------------------------------------------------------------------------
AUTO_
-----
JOB_MONTHLY_PAYROLL                                                 TRUE  SCHEDULED
PROG_MONTHLY_PAYROLL
SCH_FIRST_OF_MONTH
FALSE
```


Task3.

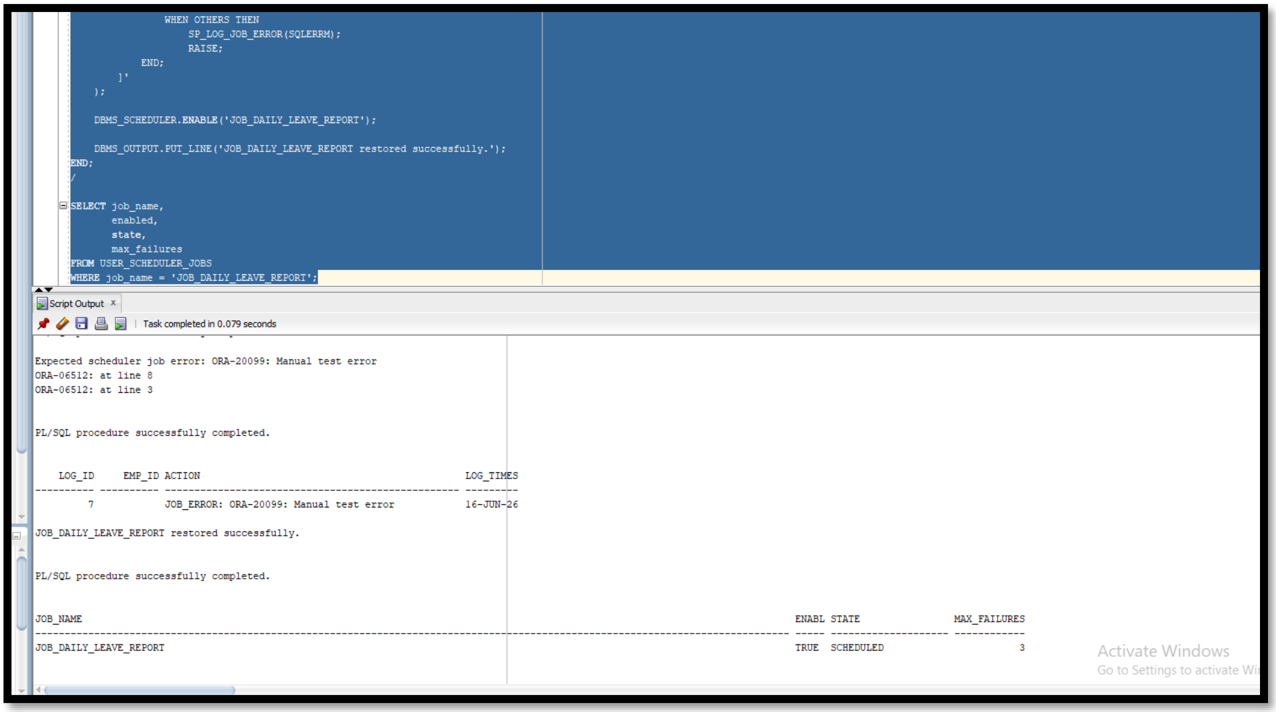
```
            EXCEPTION
                WHEN OTHERS THEN
                    SP_LOG_JOB_ERROR(SQLERRM);
                    RAISE;
            END;
        ]'
    );

    -- Set max failures to 3
    DBMS_SCHEDULER.SET_ATTRIBUTE(
        name      => 'JOB_DAILY_LEAVE_REPORT',
        attribute => 'max_failures',
        value     => 3
    );

    -- Enable job again
    DBMS_SCHEDULER.ENABLE('JOB_DAILY_LEAVE_REPORT');
```

Script Output

Task completed in 0.195 seconds

```
PROG_MONTHLY_PAYROLL
SCH_FIRST_OF_MONTH
FALSE



Procedure SP_LOG_JOB_ERROR compiled

JOB_DAILY_LEAVE_REPORT updated with error handling.
MAX_FAILURES set to 3.


PL/SQL procedure successfully completed.


JOB_NAME                                                            ENABL STATE               MAX_FAILURES
------------------------------------------------------------------------------- ----- -------------------- ------------
REPEAT_INTERVAL
--------------------------------------------------------------------------------
JOB_DAILY_LEAVE_REPORT                                              TRUE  SCHEDULED                    3
FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0
```


Task4.


WHEN OTHERS THEN
SP_LOG_JOB_ERROR(SQLERRM);
RAISE;
END;
]'
);
DBMS_SCHEDULER.ENABLE('JOB_DAILY_LEAVE_REPORT');
DBMS_OUTPUT.PUT_LINE('JOB_DAILY_LEAVE_REPORT restored successfully.');
END;
/
SELECT job_name,
enabled,
state,
max_failures
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_DAILY_LEAVE_REPORT';
Script Output
Task completed in 0.079 seconds
Expected scheduler job error: ORA-20099: Manual test error
ORA-06512: at line 8
ORA-06512: at line 3
PL/SQL procedure successfully completed.
LOG_ID EMP_ID ACTION LOG_TIMES
7 JOB_ERROR: ORA-20099: Manual test error 16-JUN-26
JOB_DAILY_LEAVE_REPORT restored successfully.
PL/SQL procedure successfully completed.
JOB_NAME ENABL STATE MAX_FAILURES
JOB_DAILY_LEAVE_REPORT TRUE SCHEDULED 3
Activate Windows